

Simple Linear Regression

Supervised ML $\rightarrow$ Regression

Dataset $\rightarrow$ I/P features

$\uparrow$ X
Weight

74

80

75

-

$\uparrow$ y O/P or
Height dependent
feature

170cm

180cm

175.5cm

-

New
weight

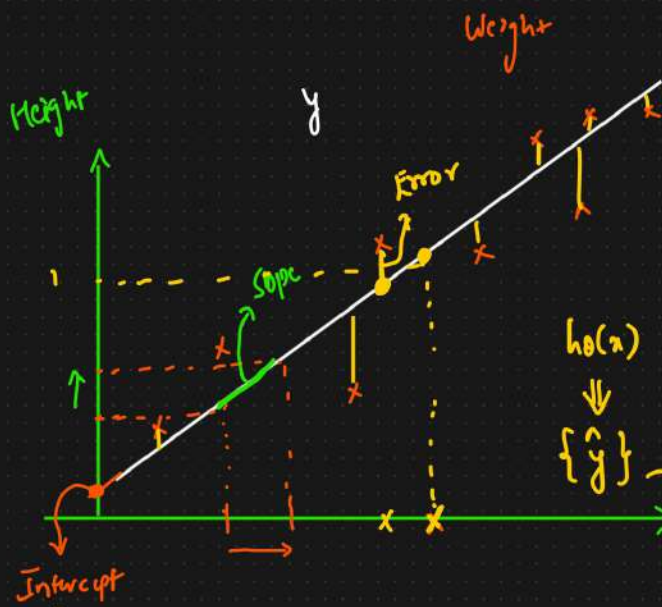
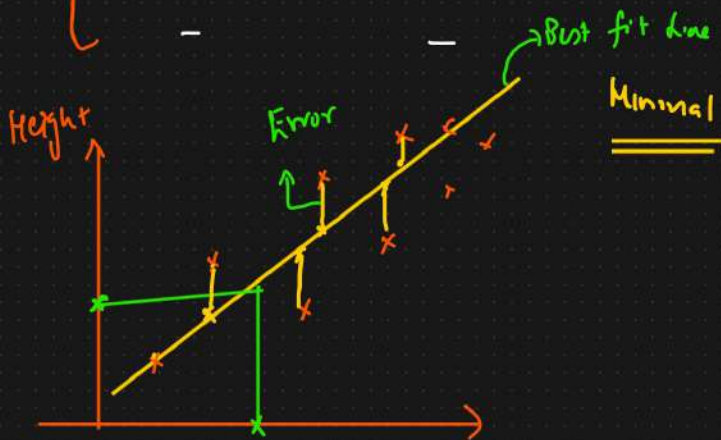
TRAIN



Model

Simple Linear
Regression

Height



$$y = mx + c$$

$$y = \beta_0 + \beta_1 x$$

$$h_0(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \dots + \theta_n x_n$$

$h_0(x)$

$\downarrow$
 $\{\hat{y}\}$

$$h_0(x) = \theta_0 + \theta_1 x$$

predicted point

if $x = 0$

$$\text{Error } (y - \hat{y})$$

θ_0 = Intercept

θ_1 = slope or coefficient

$$h_0(x) = \theta_0$$

Cost function

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (\overset{\text{predicted}}{\uparrow} h_0(x)^{(i)} - \overset{\text{True O/P}}{\uparrow} y^{(i)})^2 \Rightarrow \text{Mean Squared Error}$$

$\downarrow$ Error

Final Aim what we need to solve

Minimize $J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_0(x)^i - y^{(i)})^2$ ↓↓↓

θ_0, θ_1

① $h_0(x) = \theta_0 + \theta_1 x$

$\theta_0 = 0$

$h_0(x) = \theta_1 x$

DATASET

X	y
1	1
2	2
3	3



$\theta_0 = 0$

$h_0(x) = \theta_1 x$

let $\theta_1 = 1$ {slope}

$h_0(x) = 1$ if $x = 1$

$h_0(x) = 2$ if $x = 2$

$h_0(x) = 3$ if $x = 3$

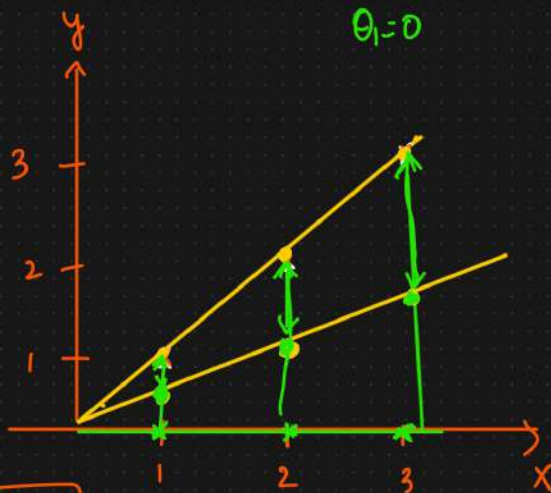
$h_0(x) = \theta_1 x$

let $\theta_1 = 0.5$

$h_0(x) = 0.5$ if $x = 1$

$h_0(x) = 1$ if $x = 2$

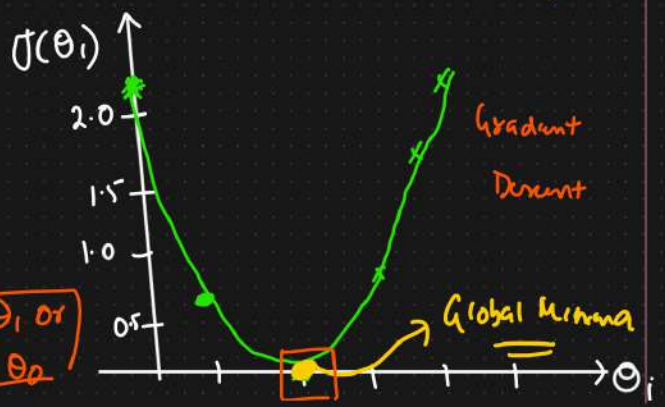
$h_0(x) = 1.5$ if $x = 3$



$\theta_1 = 1$

$$J(\theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_0(x)^{(i)} - y^{(i)})^2$$

$$= \frac{1}{2 \times 3} [(1-1)^2 + (2-2)^2 + (3-3)^2]$$



θ_1 or θ_0

$$\underline{J(\theta_1)} = 0 \leftarrow \theta_1 = 0.5$$

Error has been $0.5 \quad 1 \quad 1.5 \quad 2.0 \quad 2.5$
minimized

$$J(\theta_1) = \frac{1}{2 \times 3} \left[(0.5-1)^2 + (1-2)^2 + (1.5-3)^2 \right]$$

$$J(\theta_1) = \approx \underline{0.58} \quad \text{if } \theta_1 = 0$$

$$J(\theta_1) = \frac{1}{2 \times 3} \left[(0-1)^2 + (0-2)^2 + (0-3)^2 \right]$$

$$J(\theta_1) \approx \underline{2.3}$$

Convergence Algorithm { Optimize the changes of θ_1 values }

Repeat until convergence

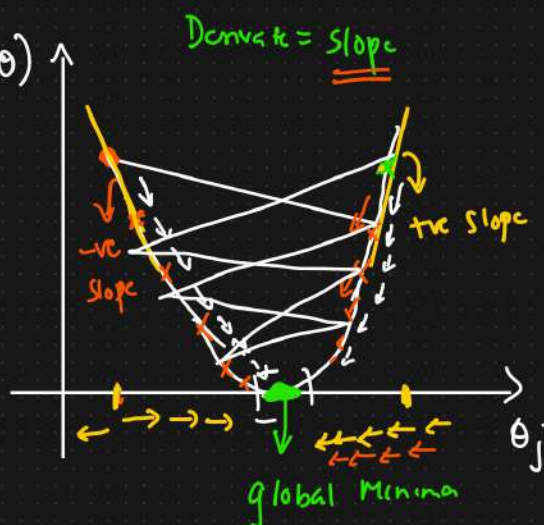
$$\left\{ \theta_j := \theta_j - \alpha \left[\frac{\partial J(\theta_j)}{\partial \theta_j} \right] \rightarrow -ve \right.$$

$$\begin{aligned} \theta_j &= \theta_j - \alpha (+ve) \\ &= \theta_j - (+ve) \end{aligned}$$

$\alpha = \text{Learning Rate}$

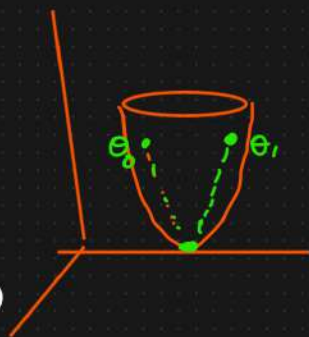
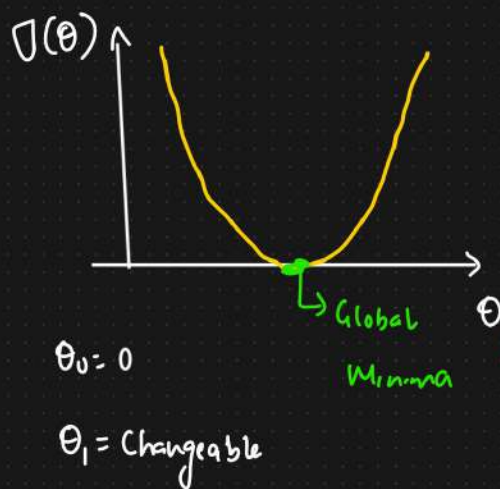
$$\left\{ \begin{aligned} \theta_j &= \theta_j - \alpha (-ve) \\ \theta_j &= \theta_j + (+ve) \end{aligned} \right.$$

$$\underline{\underline{\alpha = 0.001}} \leftarrow$$



Final Conclusion

GRADIENT DESCENT



Convergence Algorithm

repeat until convergence

{

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$$

}

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

$j = 0 \text{ and } 1$

$$\frac{\partial}{\partial x} x^2 = 2x$$
$$\frac{\partial}{\partial x} x^h = h x^{h-1}$$

$$\rightarrow \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) = \frac{\partial}{\partial \theta_j} \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

if

$$h_{\theta}(x) = \theta_0 + \theta_1 x \rightarrow 0$$

$$j=0 \Rightarrow \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1) = \frac{\partial}{\partial \theta_0} \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \right]$$
$$= \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \times 1$$

$$j=1 \Rightarrow \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1) = \frac{\partial}{\partial \theta_1} \left[\frac{1}{2m} \sum_{i=1}^m ((\theta_0 + \theta_1 x) - y^{(i)})^2 \right]$$

$$= \frac{1}{m} \sum_{i=1}^m ((\theta_0 + \theta_1 x) - y^{(i)}) (x)$$

$$\frac{\partial}{\partial \theta_1} [\theta_0 + \theta_1 x] \Rightarrow x =$$

Repeat until convergence

$$\left\{ \begin{array}{l} \theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x)^{(i)} - y^{(i)}) \\ \theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x)^{(i)} - y^{(i)}) x^{(i)} \end{array} \right\}$$

Multiple Linear Regression

Dataset

Weight $\swarrow$ I/P $\searrow$ O/P feature
Height

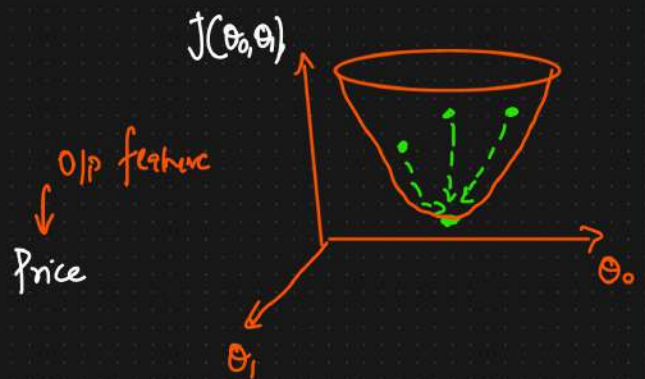
$$h_0(x) = \theta_0 + \theta_1 x \quad \swarrow \text{I/P or Independent}$$

θ_0 = Intercept

θ_1 = Slope

House Price Dataset

No. of Rooms $\swarrow$ Independent features $\searrow$ Price
Size of House
Location



$$h_0(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 \quad \left\{ \begin{array}{l} \text{Multiple Linear} \\ \text{Regression} \end{array} \right.$$

$\theta_1, \theta_2, \theta_3$ = coefficient

θ_0 = Intercept

Performance Metrics Used In Linear Regression

① R squared

② Adjusted R squared

$$R_{\text{squared}} = 1 - \frac{SS_{\text{Res}}}{SS_{\text{Total}}}$$

$$= 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y}_i)^2}$$

$$= 1 - \frac{\text{Small number}}{\text{Bigger number}}$$

$$= 1 - \text{Small number}$$

$$\approx 1$$

$$0.70 \Rightarrow 70\%$$

$$0.85 \Rightarrow 85\%$$

$$0.90 \Rightarrow 90\%$$

{ Overfitting, Underfitting }

→ 1

↓

Accuracy My
Model Is?

R squared ↑↑↑

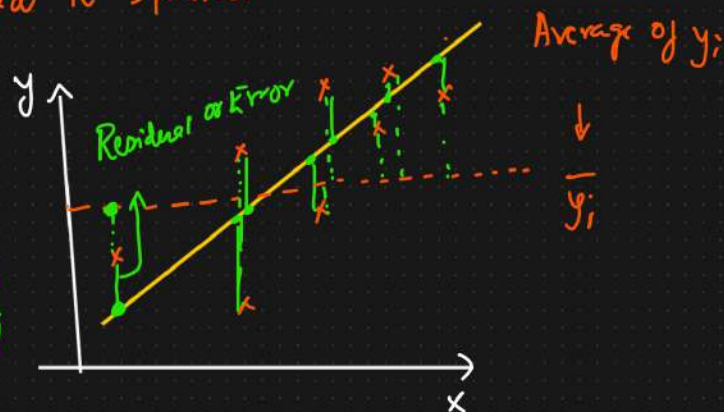
Size of the house ↑ Price ↑

+ve correlation

No. of bedrooms ↑ Price ↑

+ve correlation

This is the problem of R squared



② Adjusted R squared

Datant

→ (Price)

↓
Gender | Size of the house | No. of bedrooms | Location | Price

$$R_{\text{squared}} = 75\% \Rightarrow 0.75$$

$$R_{\text{squared}} \Rightarrow 80\% \Rightarrow 0.80$$

$$R_{\text{squared}} \Rightarrow 85\% \Rightarrow 0.85$$

$$R_{\text{squared}} \Rightarrow 87\% \Rightarrow 0.87$$

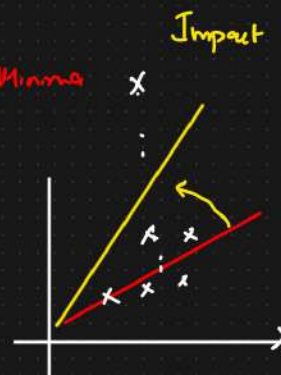
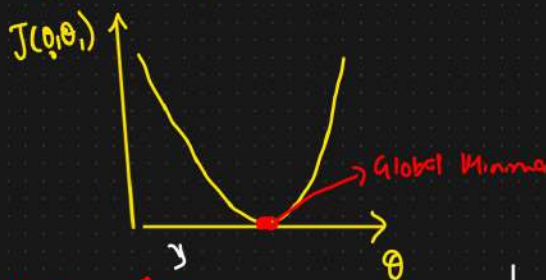
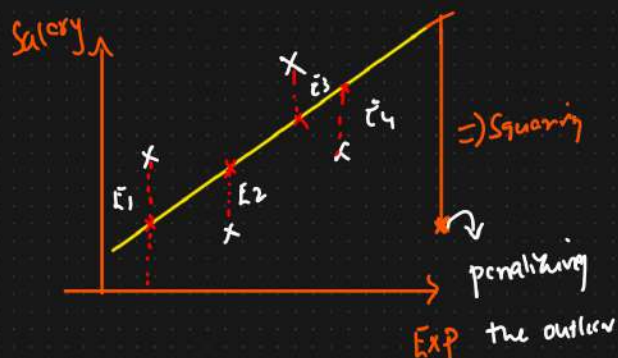
$$\text{Adjusted } R_{\text{squared}} = 1 - \frac{(1-R^2)(N-1)}{N-p-1} \quad \left\{ \begin{array}{ll} p=2 & R^2=90\% \quad R^2_{\text{adjusted}}=86\% \\ p=3 & R^2=92\% \quad R^2_{\text{adjusted}}=82\% \end{array} \right.$$

N = No. of data points

p = No. of Independent features

MSE, MAE, RMSE [Cost function] → Performance Metrics

R^2 and Adjusted R^2



MSE $\uparrow \uparrow$

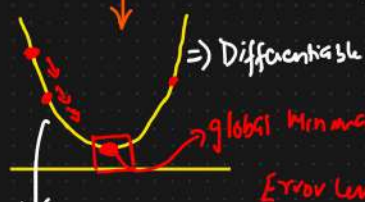
MSE

$$MSE = \sum_{i=1}^n \frac{(y_i - \hat{y}_i)^2}{n} \Rightarrow \text{Cost function} \downarrow$$

Quadratic Equation

$$ax + by + c$$

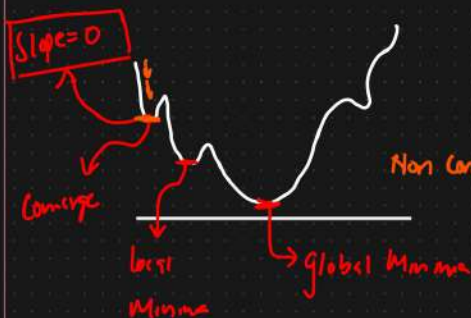
$$(a-b)^2 = a^2 - 2ab + b^2$$



Convex function

- Advantage
- ① Differentiable ✓
 - ② It has one local and one global Minima
 - ③ Converges faster

- Disadvantage
- ④ Not Robust to outliers
 - ⑤ It is not in same unit.



$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Salary (lakh)

x y $(y_i - \hat{y}_i)^2$ (lakh)² (MSE)

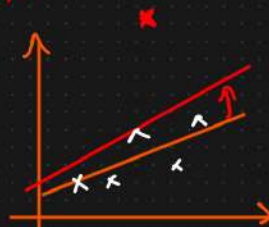
$$\text{Error } 2.5 \Rightarrow (\text{lakh})^2 \Leftarrow$$

② Mean Absolute Error (MAE)

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

lakh

MAE $\uparrow$

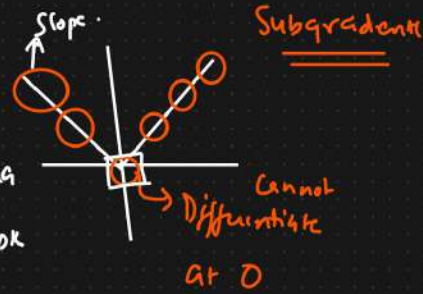


Advantage

- ① Robust to outliers ✓
- ② It will be in the same unit

Disadvantage

- ④ Convergence usually take more time. Optimization is a complex task
- ⑤ Time consuming

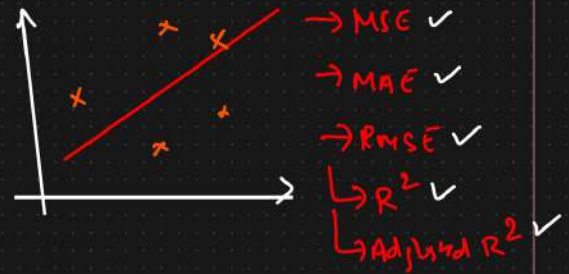


③ RMSE {Root Mean Squared Error}

MAE, MSE, RMSE

Performance
↑ metric

$$\text{RMSE} = \sqrt{\text{MSE}}$$
$$= \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$



Advantage

- ① Same unit
- ② Differentiable

Disadvantage

- ④ ^{Not} Robust to outliers

MSE vs MAE vs RMSE }
R² vs Adjusted R² }

Overfitting And Underfitting (Bias And Variance)

- ① Training dataset
- ② Test dataset
- ③ Validation dataset

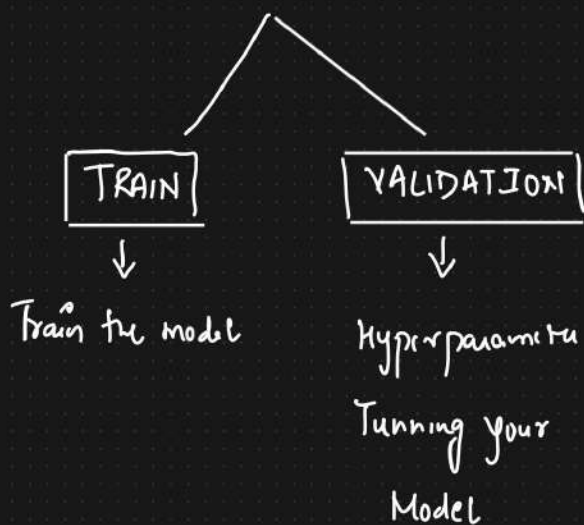
For 30%

Dataset
1000 datapoints

→ TRAINING DATASET (700) → Model To TRAIN
→ TEST DATASET (300) → Testing Model

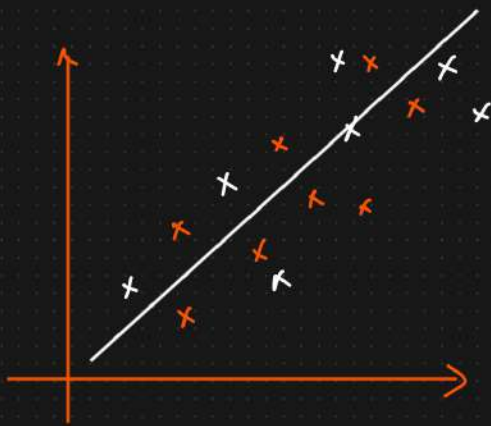
Size of house	No. of bedrooms	Price
—	—	—
—	—	—
—	—	—
—	—	—

TRAINING DATASET



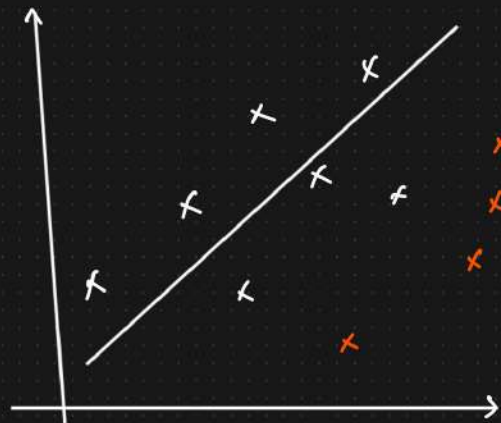
{	TRAIN	(90%) Very Good Accuracy [Low Bias]	Very Good Accuracy (90%) [Low Bias]
	TEST	Very Good Accuracy (85%) [Low Variance] ↑	Bad Accuracy (50%) [High Variance] ↓
		→ Generalized Model	Model is Overfitting

TRAIN Model Accuracy is low [High Bias]
 TEST Model Accuracy is low [High Variance]
 ↓
 Model is Underfitting



Generalized Model

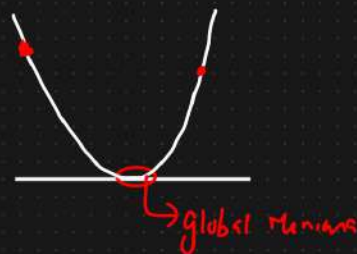
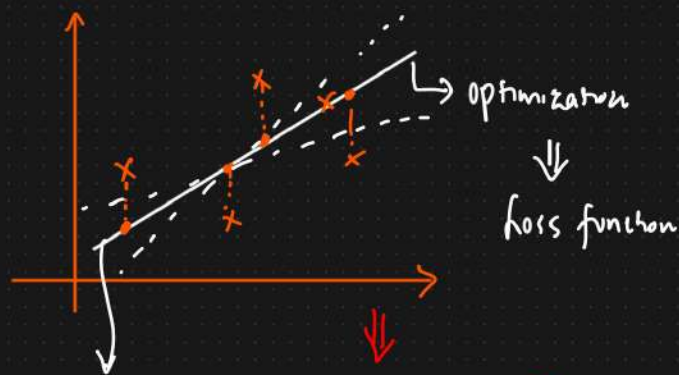
↓
Low Bias, Low Variance



Overfitting

Low Bias, High Variance

Linear Regression Using OLS {Ordinary Least Square}



$h_0(x) = \beta_0 + \beta_1 x$
 $\hookrightarrow \hat{y}$

$\rightarrow$ OLS $\rightarrow$ Formula and Calculate
 β_0 & β_1
 $=$ $=$

Ordinary Least Square

$$S(\beta_0, \beta_1) = \frac{1}{n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$$

$\Downarrow$
 Find β_0 & β_1
 $=$ $=$

$$\begin{aligned} \frac{\partial S}{\partial \beta_0}(\beta_0, \beta_1) &= \frac{2}{n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) (0 - 1 - 0) \\ &= -\frac{2}{n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) = 0 \rightarrow (1) \end{aligned}$$

$$\begin{aligned} \frac{\partial S}{\partial \beta_1}(\beta_0, \beta_1) &= \frac{2}{n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) (-x_i) = 0 \rightarrow (2) \\ &= -\frac{2}{n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) (x_i) = 0 \rightarrow (2) \end{aligned}$$



$$y = x^2$$

$$\begin{aligned} \frac{\partial y}{\partial x} &= 2(x)^{2-1} \frac{\partial (x)}{\partial x} \\ &= 2x \end{aligned}$$

$$\varepsilon_q \rightarrow \textcircled{1}$$

$$-\frac{2}{n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) = 0$$

$$\Downarrow$$

$$-\sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) = 0$$

$$-\sum_{i=1}^n y_i + n\beta_0 + \beta_1 \sum_{i=1}^n x_i = 0$$

$$n\beta_0 + \beta_1 \sum_{i=1}^n x_i = \sum_{i=1}^n y_i$$

$$\beta_0 = \frac{\sum_{i=1}^n y_i}{n} - \beta_1 \frac{\sum_{i=1}^n x_i}{n}$$

Intercept



$$\boxed{\beta_0 = \bar{y} - \beta_1 \bar{x}}$$

$$\boxed{\varepsilon_q, 2 \div}$$

$$-\frac{2}{n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)(x_i) = 0$$

$$\Downarrow$$

$$\sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)(x_i) = 0$$

$$\Downarrow$$

$$\sum_{i=1}^n x_i y_i - \beta_0 \sum_{i=1}^n x_i - \beta_1 \sum_{i=1}^n x_i^2 = 0$$

$$\sum_{i=1}^n (x_i y_i - \beta_0 x_i - \beta_1 x_i^2) = 0$$

↓

Replace $\beta_0 = \bar{y} - \beta_1 \bar{x}$

$$\sum_{i=1}^n (x_i y_i - (\bar{y} - \beta_1 \bar{x}) x_i - \beta_1 x_i^2) = 0$$

$$\sum_{i=1}^n (x_i y_i - x_i \bar{y} + \beta_1 \bar{x} x_i - \beta_1 x_i^2) = 0$$

$$\sum_{i=1}^n (y_i - \bar{y} + \beta_1 \bar{x} - \beta_1 x_i) x_i = 0$$

$$\sum_{i=1}^n [(y_i - \bar{y}) + \beta_1 (\bar{x} - x_i)] = 0$$

$$\sum_{i=1}^n (y_i - \bar{y}) + \sum_{i=1}^n \beta_1 (\bar{x} - x_i) = 0$$

$$\sum_{i=1}^n \beta_1 (\bar{x} - x_i) = - \sum_{i=1}^n (y_i - \bar{y})$$

$$\beta_1 = \frac{- \sum_{i=1}^n (y_i - \bar{y})}{\sum_{i=1}^n (\bar{x} - x_i)}$$

$$\beta_1 = \frac{\sum_{i=1}^n (y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})}$$

Coefficient $\Rightarrow$

$$\beta_1 = \frac{\sum_{i=1}^n (y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})}$$

Intercept
↓

Ols

↑
coefficient

$$\boxed{\beta_0 = \bar{y} - \beta_1 \bar{x}}$$

coefficient
↓

Intercept
↓

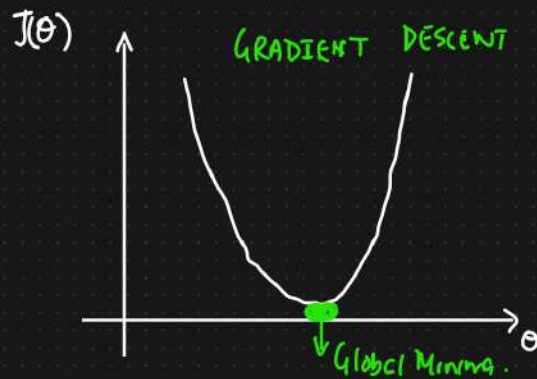
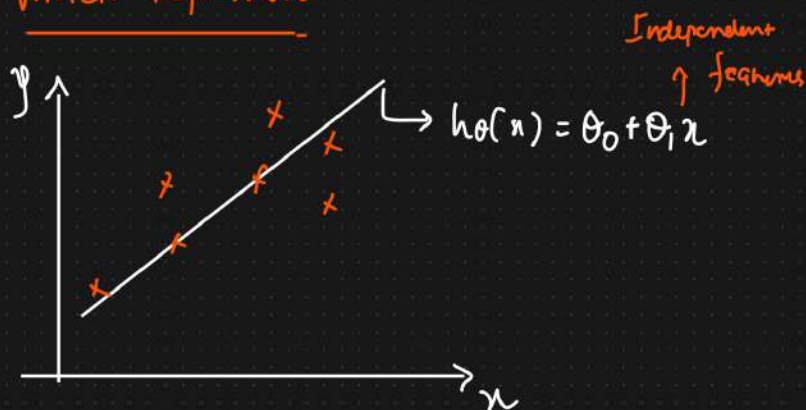
$$\beta_0 = (\bar{y} - \beta_1 \bar{x})$$

x	y	$(y_i - \bar{y}) \div (x_i - \bar{x})$	
-	-	-	-
-	-	-	-
-	-	-	-
-	-	-	-
$\bar{x}$	$\bar{y}$		

Ols $\approx$ Linear Regression (sklearn)

Ridge Regression, Lasso Regression, ELasticnet Regression

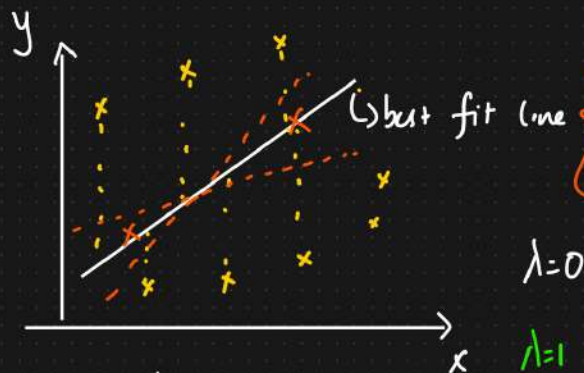
Linear Regression



$$\text{Cost fn} = \frac{1}{2m} \sum_{i=1}^m (h_0(x^{(i)}) - y^{(i)})^2$$

Mean Squared Error

① Ridge Regression (L2 Regularization) $\rightarrow$ Reduce Overfitting



Train data $\rightarrow$ Acc $\uparrow$ $\rightarrow$ low Bias
Test data $\rightarrow$ Acc $\downarrow$ $\rightarrow$ High Variance

$\lambda = 0$

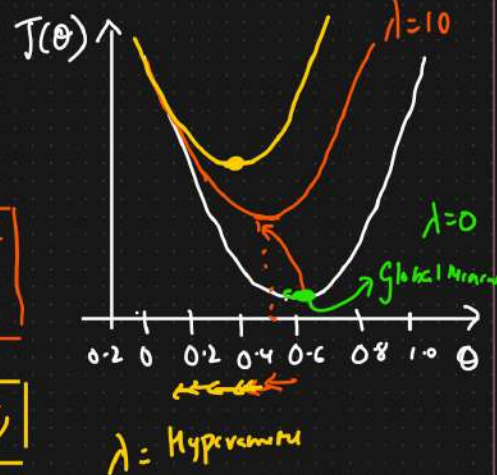
$\lambda = 1$

$\lambda = 10$

Hyperparameter

$$\text{Cost fn} = \frac{1}{2m} \sum_{i=1}^m (h_0(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{i=1}^m (\text{slope})^2$$

$\lambda \uparrow$ slope $\downarrow$



$\lambda = 1$

> 0

$$h_0(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$$

$\theta_0 = 0.34$
 $\theta_1 = 0.52$
 $\theta_2 = 0.48$
 $\theta_3 = 0.24$



$$= 0.34 + 0.40x_1 + 0.38x_2 + \boxed{0.14x_3}$$

↑↑↑

② Lasso Regression (L_1 Regularization) $\rightarrow$ Feature Selection

$$\text{Cost fn} = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{i=1}^n |\text{slope}|$$

↑↑

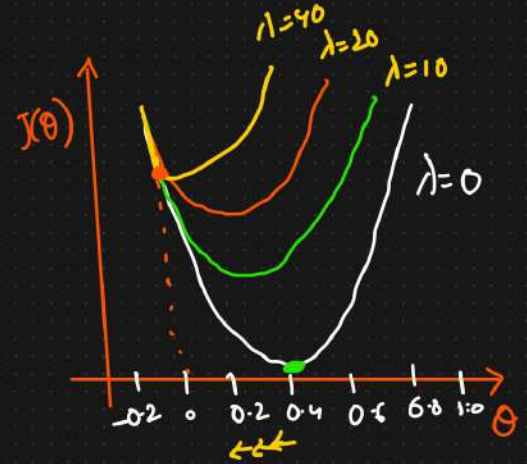
$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \theta_4 x_4$$

$$h_{\theta}(x) = 0.52 + 0.65x_1 + 0.72x_2 + 0.34x_3 + \boxed{0.12x_4}$$

$\Downarrow$
Lasso Regression

$$= 0.52 + 0.51x_1 + 0.60x_2 + 0.14x_3 + \boxed{0x_4}$$

$\nearrow 0$



③ ElasticNet Regression

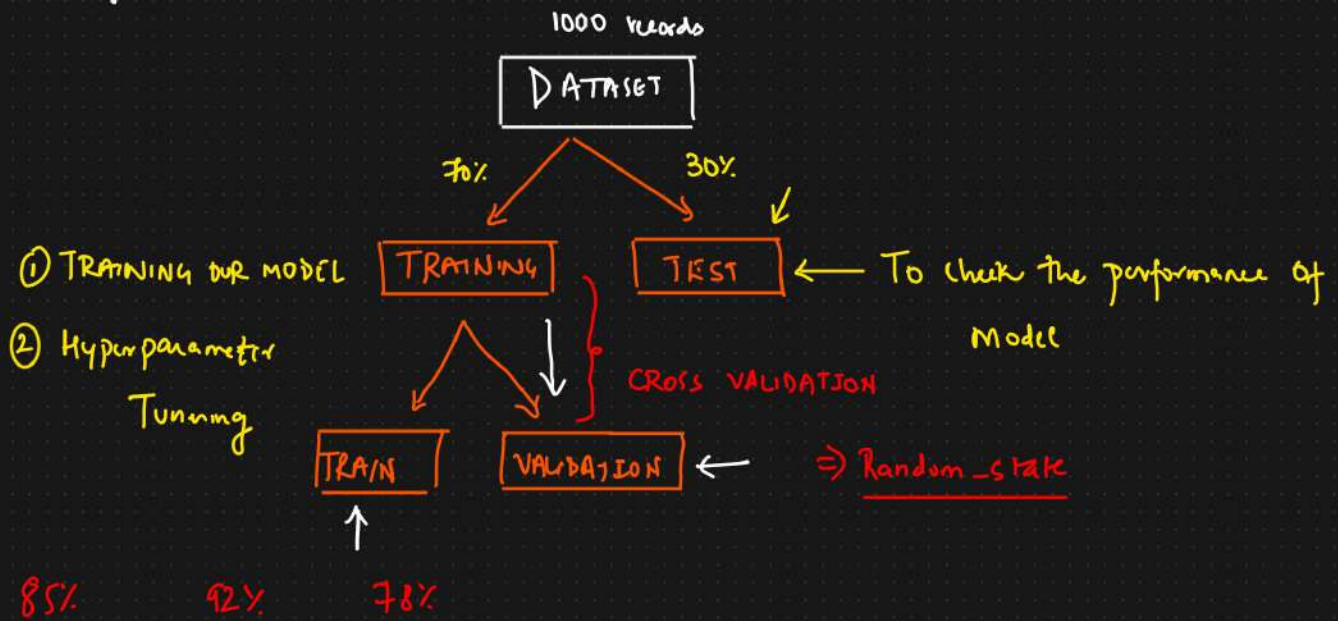
- $\rightarrow$ ① Reduce Overfitting
- $\rightarrow$ ② Feature Selection

$$\text{Cost fn} = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda_1 \sum_{i=1}^m (\text{slope})^2 + \lambda_2 \sum_{i=1}^m |\text{slope}|$$

$\Downarrow$ Reduce Overfitting $\Downarrow$ Feature Selection

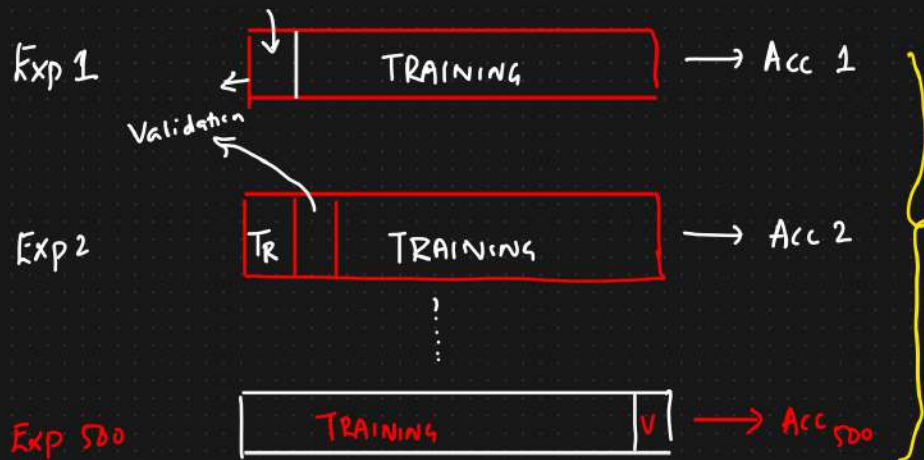
Hyperparameter Tuning the Linear Regression

Types of CROSS VALIDATION



- ① leave One Out CV (LOOCV) ② leave p out CV
- P=10 P=20 P=30

TRAINING → 500 Records ↑↑ Complexity of Training Model



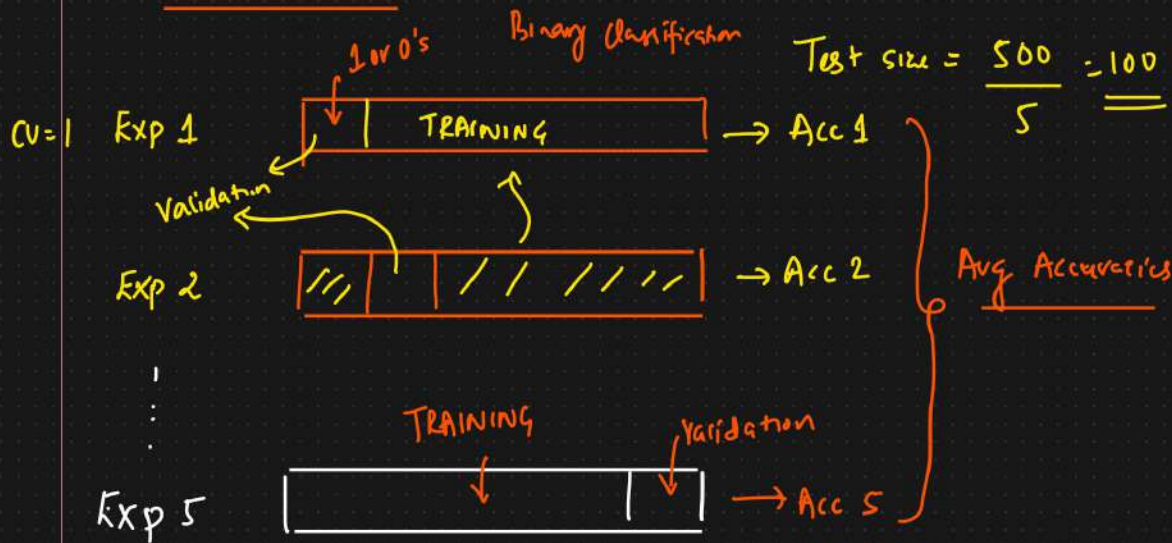
① Overfitting → **TRAINING** ↑↑ Acc → New Test Data → Acc ↓↓

Validation Acc ↓↓

③ K Fold CV

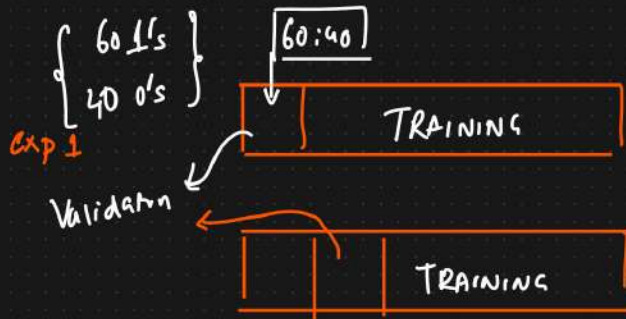
$K=5$

$n=500$



④ Stratified K Fold CV

$K=5$



Test size = $\frac{500}{5} = 100$

↓

Validation Data

⑤ Time Series CV

Reviews

Product Sentiment Analysis

Time

JAN → DEC

DAY 1 DAY 2 DAY 3 DAY 4 | DAY N

TRAINING

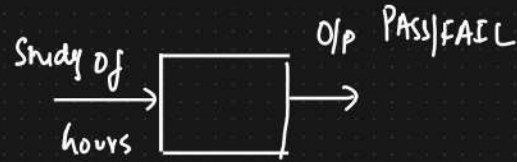
Validation

Time Series Application

Logistic Regression (Binary classification) ←

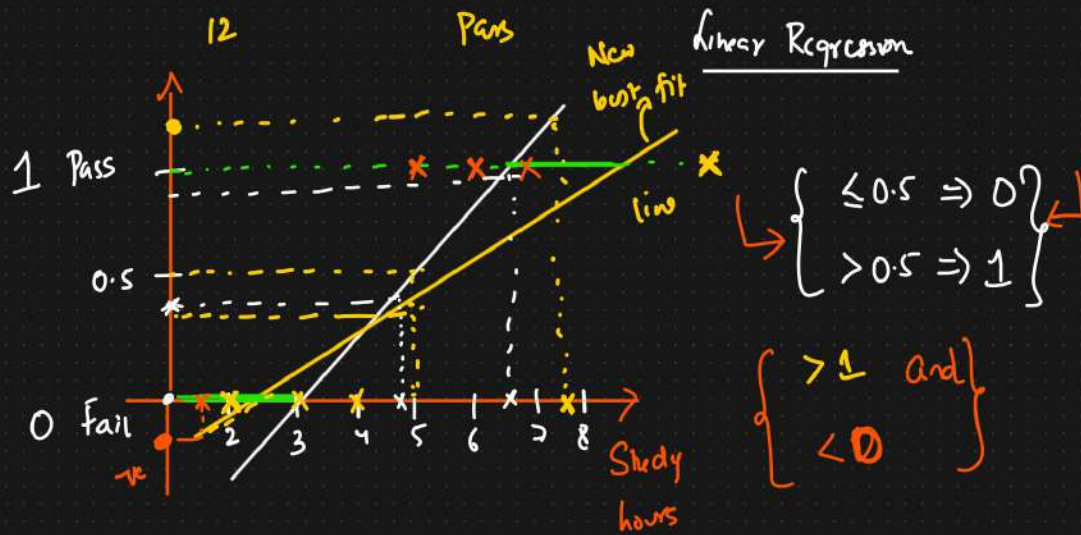
<u>Dataset</u>	<u>Pass/Fail</u>
Study hours	O/p {Binary categories}

2	Fail
3	Fail
4	Fail
5	Pass
6	Pass
7	Pass
12	Pass



0 to 1

Logistic Regression

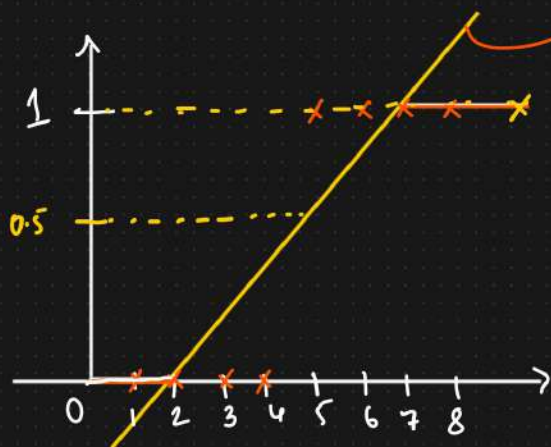


Why we cannot use Linear Regression for Classification?

① Outlier {Best fit line change}

② > 1 and < 0 {Squash line}

How Logistic Regression Solves Classification Problem

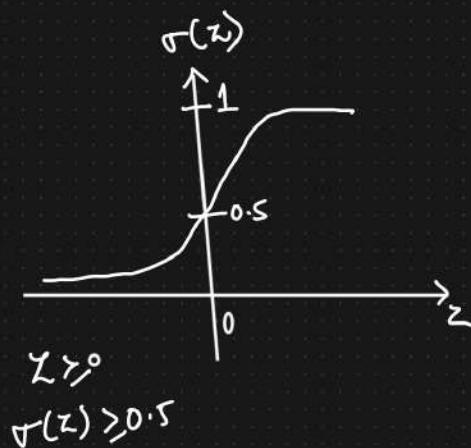


$$h_0(x) = \theta_0 + \theta_1 x_1$$

↓
Best fit line

Sigmoid Activation

$$\frac{1}{1 + e^{-z}}$$



$$h_0(x) = \sigma(\theta_0 + \theta_1 x_1)$$

$$\sigma = \frac{1}{1 + e^{-z}}$$

$$= \sigma(z)$$

$$h_0(x) = \frac{1}{1 + e^{-z}}$$

⇒ Logistic Regression hypothesis

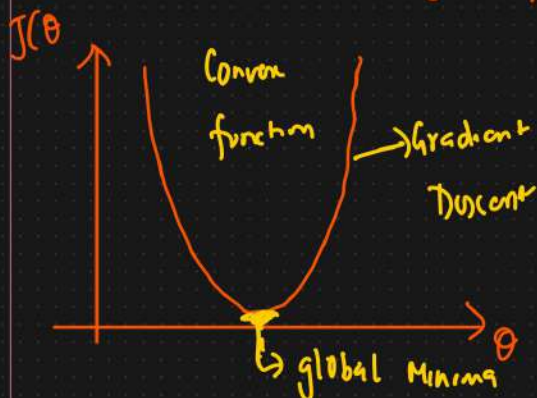
$$z = \theta_0 + \theta_1 x_1$$

Linear Regression Cost function

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_0(x^{(i)}) - y^{(i)})^2$$

$$h_0(x) = \theta_0 + \theta_1 x$$

↓
Convex function



Logistic Regression Cost function

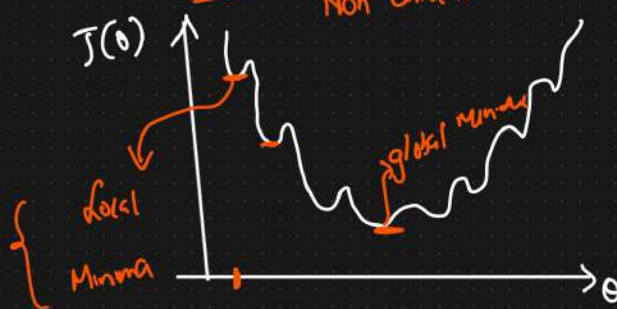
$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_0(x^{(i)}) - y^{(i)})^2$$

Non Convex function

$$h_0(x) = \frac{1}{1 + e^{-z}} \quad z = \theta_0 + \theta_1 x$$

Sigmoid

Non Convex



$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m \underbrace{(h_{\theta}(x^{(i)}) - y^{(i)})^2}_{\text{cost}(h_{\theta}(x^{(i)}), y^{(i)})}$$

$$h_{\theta}(x^{(i)}) = \frac{1}{1+e^{-z}} \quad z = \theta_0 + \theta_1 x$$

↓

this denote $\text{cost}(h_{\theta}(x^{(i)}), y^{(i)})$

{ log loss }

$$\text{cost}(h_{\theta}(x^{(i)}), y^{(i)}) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y=1 \\ -\log(1-h_{\theta}(x)) & \text{if } y=0 \end{cases}$$

↓ convex function

$$\text{cost}(h_{\theta}(x^{(i)}), y^{(i)}) = -y \log(h_{\theta}(x)) - (1-y) \log(1-h_{\theta}(x))$$

$$J(\theta_0, \theta_1) = -\frac{1}{2m} \sum_{i=1}^m (y^{(i)} \log(h_{\theta}(x^{(i)})) - (1-y^{(i)}) \log(1-h_{\theta}(x^{(i)})))$$

Minimize cost function $J(\theta_0, \theta_1)$ by changing

θ_0 & θ_1

Convergence Algorithm

Repeat

{

$j=0$ and 1

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$$

}

Performance Metrics, Accuracy, Precision, Recall And F-Beta

Topics to be covered

- ① Confusion matrix
- ② Accuracy
- ③ Precision
- ④ Recall
- ⑤ F-Beta Score



Logistic Regression

R squared
Adjusted R squared

① Confusion Matrix

	1	0	
1	3	2	
0	1	1	

Actual Values

Dataset		o/p	pred by model
x_1	x_2	y	$\hat{y}$
-	-	0	1
-	-	1	1
-	-	0	0
-	-	1	1
-	-	1	1
-	-	0	1
-	-	1	0

Predicted values

	1	0	
1	TP	FP	
0	FN	TN	

Actual

$$\begin{aligned}
 \text{Accuracy} &= \frac{TP + TN}{TP + FP + FN + TN} \\
 &= \frac{3 + 1}{3 + 2 + 1 + 1} \\
 &= \frac{4}{7}
 \end{aligned}$$

④ Dataset Binary classification

$\rightarrow$ 1000 datapoints $\left\{ \begin{array}{l} 900 \rightarrow 1 \\ 100 \rightarrow 0 \end{array} \right\}$ Imbalanced Dataset
 $\downarrow$
 90% Accuracy

④ Precision = $\frac{TP}{TP+FP}$ } Out of all the actual value
how many are correctly predicted

	1	0	Actual
1	TP	FP	
0	FN	TN	
	Predicted		

Predicted

⑤ Recall = $\frac{TP}{TP+FN}$ } Out of all the predicted value
how many are correctly predicted

Usecase ①

Spam classification

	Spam	Not Spam	
	1	0	Actual
Spam	1	TP	FP ✓
Not Spam	0	FN	TN

Mail → Spam } Good
Model → Spam }

Mail → Not Spam } Blunder
Model → Spam }

$$\text{Precision} = \frac{TP}{TP+FP}$$

Usecase 2

To predict whether person has diabetes or not

✓ Truth → diabetes }
✓ Model → Doesn't diabetes } Blunder

	Diab	No Diab	
	1	0	
Diab	1	TP	FP
No Diab	0	FN	TN

$$\text{Recall} = \frac{TP}{TP+FN}$$

Truth → diabetes } good
Model → " }

usecase of diseases

$\left. \begin{array}{l} \text{Truth} \rightarrow \text{Not diabetes} \\ \text{Model} \rightarrow \text{Diabetes} \end{array} \right\} \Rightarrow \text{2nd opinion}$

$\Downarrow$
check

Assignment

④ Tomorrow the stock market will crash or not

Reducing FPL or FN↓

$$\textcircled{v} \quad \underline{\text{F-Beta Score}} = \frac{(1 + \beta^2) \text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

① If FP & FN are both important

$$\beta = 1$$

$$F1 \text{ Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

② If FP is more important FN

$$\beta = 0.5$$

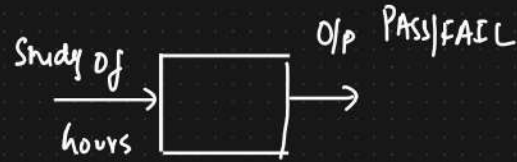
$$F_{0.5} \text{ score} = \frac{(1+0.25) \frac{P \times R}{P+R}}{P+R}$$

$$\beta = 2$$

Logistic Regression (Binary classification) ←

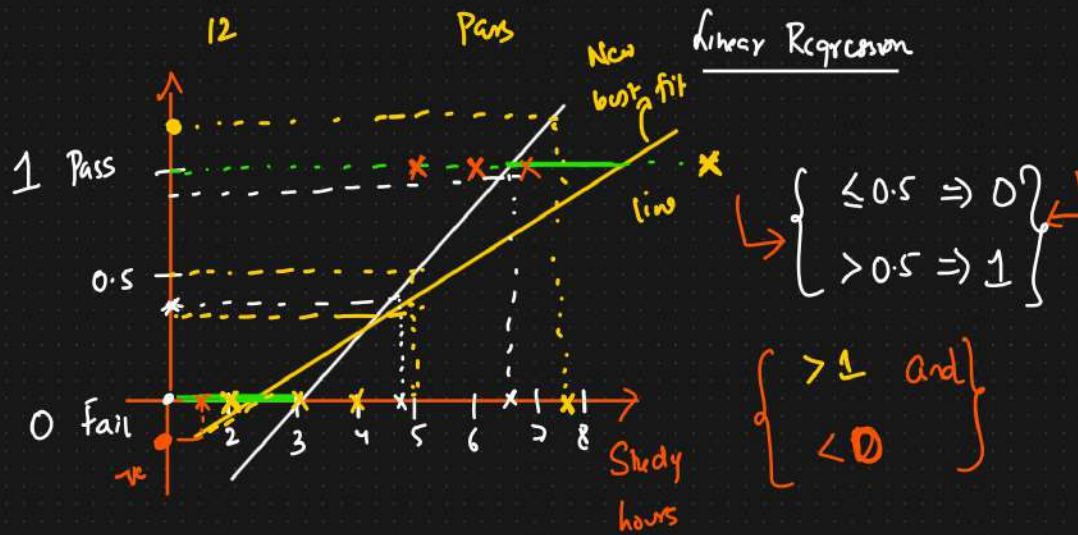
<u>Dataset</u>	<u>Pass/Fail</u>
Study hours	O/p {Binary categories}

2	Fail
3	Fail
4	Fail
5	Pass
6	Pass
7	Pass
12	Pass



0 to 1

Logistic Regression

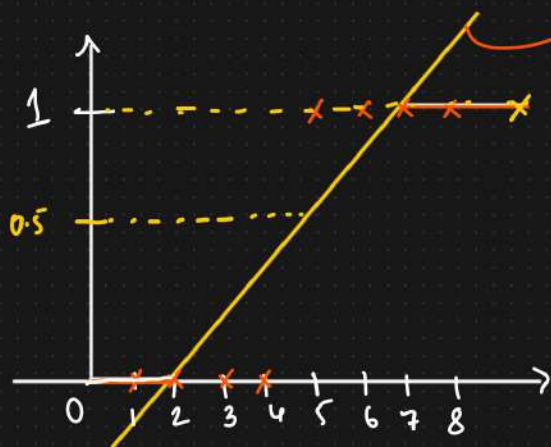


Why we cannot use Linear Regression for Classification?

① Outlier {Best fit line change}

② > 1 and < 0 {Squash line}

How Logistic Regression Solves Classification Problem

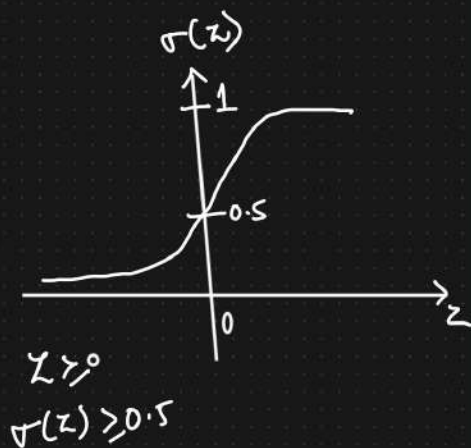


$$h_0(x) = \theta_0 + \theta_1 x_1$$

↓
Best fit line

Sigmoid Activation

$$\frac{1}{1 + e^{-z}}$$



$z > 0$
 $\sigma(z) > 0.5$

$$h_0(x) = \sigma(\theta_0 + \theta_1 x_1)$$

$$\sigma = \frac{1}{1 + e^{-z}}$$

$$= \sigma(z)$$

$$h_0(x) = \frac{1}{1 + e^{-z}}$$

⇒ Logistic Regression hypothesis

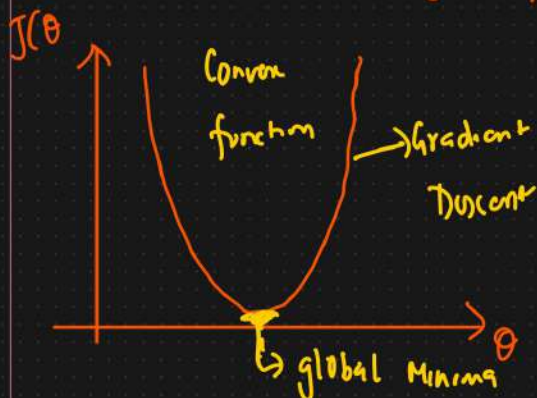
$$z = \theta_0 + \theta_1 x_1$$

Linear Regression Cost function

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_0(x^{(i)}) - y^{(i)})^2$$

$$h_0(x) = \theta_0 + \theta_1 x$$

↓
Convex function



Logistic Regression Cost function

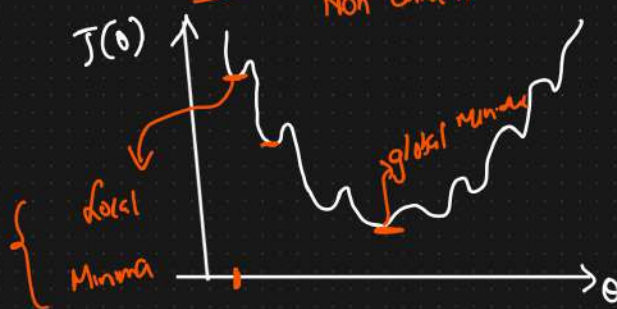
$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_0(x^{(i)}) - y^{(i)})^2$$

Non Convex function

$$h_0(x) = \frac{1}{1 + e^{-z}} \quad z = \theta_0 + \theta_1 x$$

Sigmoid

Non Convex



$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m \underbrace{(h_{\theta}(x^{(i)}) - y^{(i)})^2}_{\text{cost}(h_{\theta}(x^{(i)}), y^{(i)})}$$

$$h_{\theta}(x^{(i)}) = \frac{1}{1+e^{-z}} \quad z = \theta_0 + \theta_1 x$$

↓

this denote $\text{cost}(h_{\theta}(x^{(i)}), y^{(i)})$

{ log loss }

$$\text{cost}(h_{\theta}(x^{(i)}), y^{(i)}) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y=1 \\ -\log(1-h_{\theta}(x)) & \text{if } y=0 \end{cases}$$

↓ convex function

$$\text{cost}(h_{\theta}(x^{(i)}), y^{(i)}) = -y \log(h_{\theta}(x)) - (1-y) \log(1-h_{\theta}(x))$$

$$J(\theta_0, \theta_1) = -\frac{1}{2m} \sum_{i=1}^m (y^{(i)} \log(h_{\theta}(x^{(i)})) - (1-y^{(i)}) \log(1-h_{\theta}(x^{(i)})))$$

Minimize cost function $J(\theta_0, \theta_1)$ by changing

θ_0 & θ_1

Convergence Algorithm

Repeat

{

$j=0$ and 1

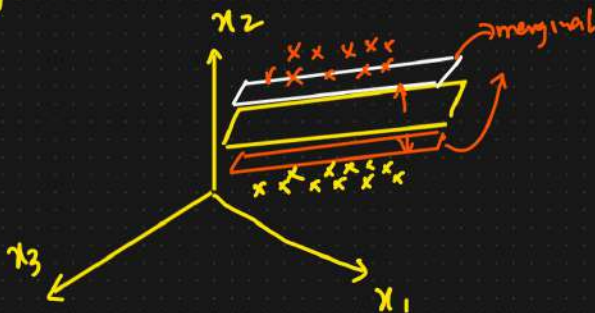
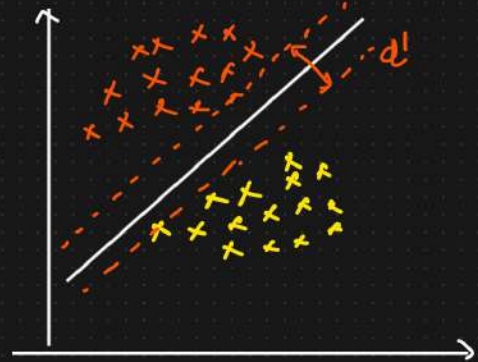
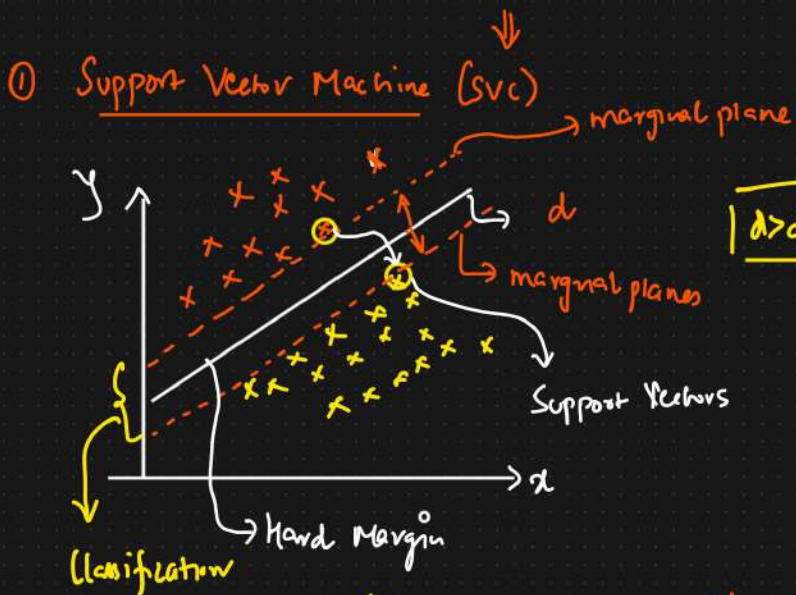
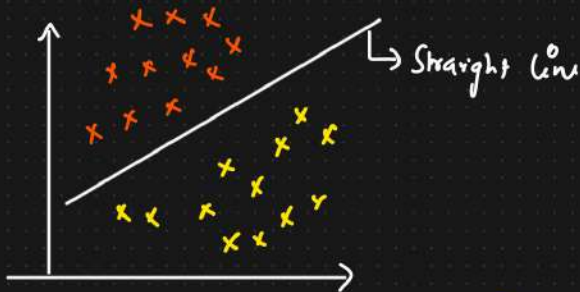
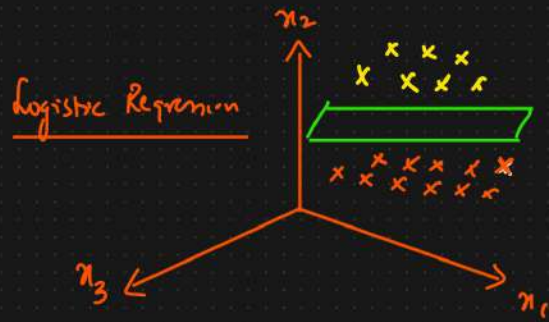
$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$$

}

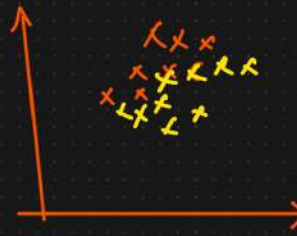
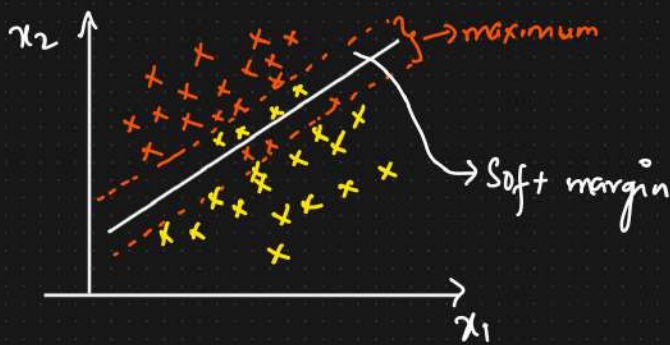
Support Vector Machines ML Algorithms.

① SVC (Support Vector Classifier)

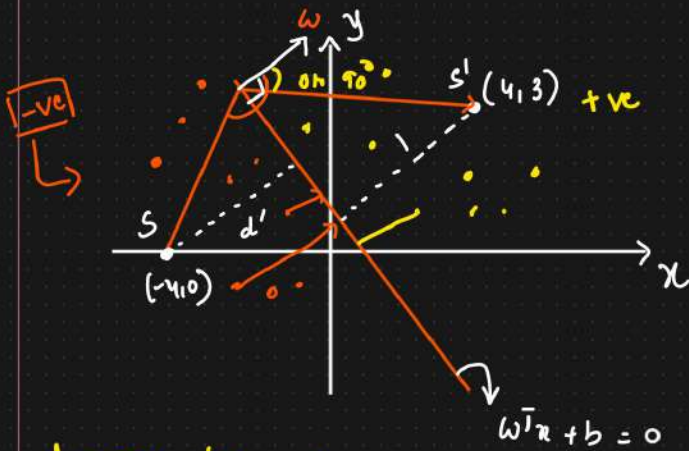
② SVR (Support Vector Regressor)



Soft Margin And Hard Margin In SVM



Support Vector Machines (SVC) Maths Intuition



$$ax + by + c = 0$$

↓

$$w_1 x_1 + w_2 x_2 + b = 0$$

$$\boxed{w^T x + b = 0}$$

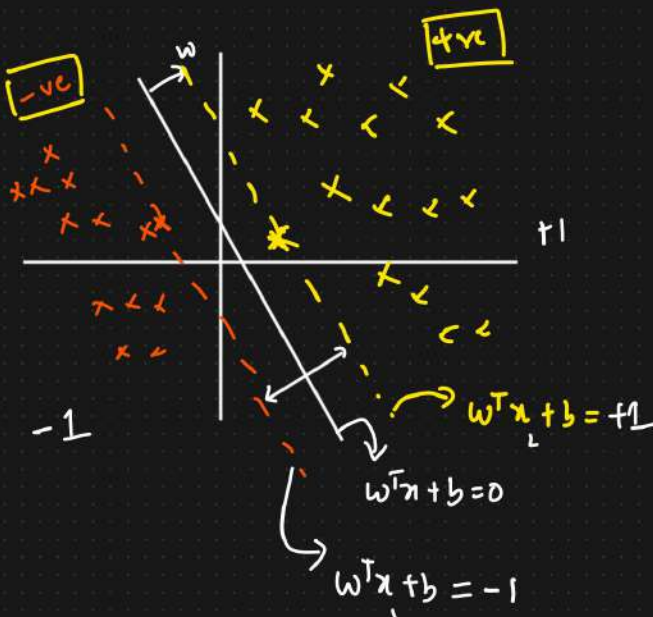
↓

$$b = 0$$

$$\boxed{w^T x = 0}$$

$d = -ve$ below plane

$d = +ve$ above plane



$$w^T x_1 + b = 1$$

$$w^T x_2 + b = -1$$

$$\begin{matrix} (-) & (-) & (+) \end{matrix}$$

$$\frac{w^T (x_1 - x_2)}{\|w\|} = \frac{+2}{\|w\|}$$

Unit vector { Magnitude of the vector is 1 }

Cost function

Maximize $\frac{2}{\|w\|}$ $\Rightarrow$ Distance between Marginal plane
 w, b

Constraint such that

$$y_i \begin{cases} +1 & w^T x + b \geq 1 \\ -1 & w^T x + b \leq -1 \end{cases}$$

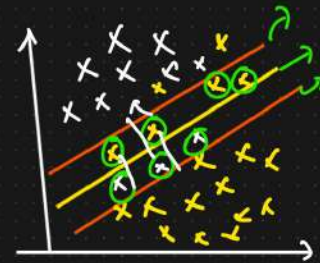
For all correct points

Constraint $\rightarrow y_i (w^T x + b) \geq 1$

Maximize $\frac{2}{\|w\|}$
 w, b

$\Rightarrow \min_{(w,b)} \frac{\|w\|}{2}$

$C_i = 6$ ✓



Cost function of SVM (svc)

Min $\frac{\|w\|}{2}$
 w, b

$+ \sum_{i=1}^n C_i \xi_i$

Hinge Loss

$\Downarrow$

Soft margin

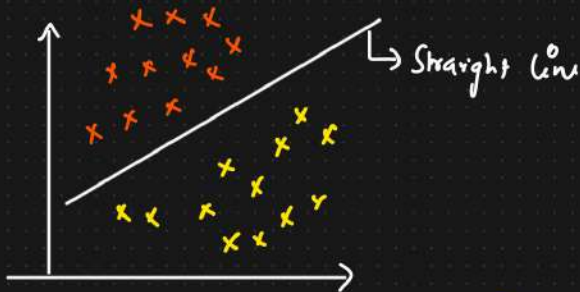
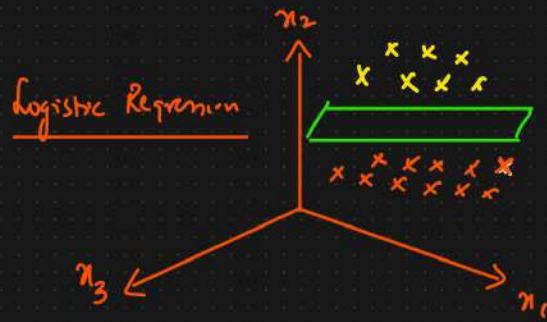
{ how many points we want to avoid misclassification }

{ distance of the incorrect data points from the marginal plane }

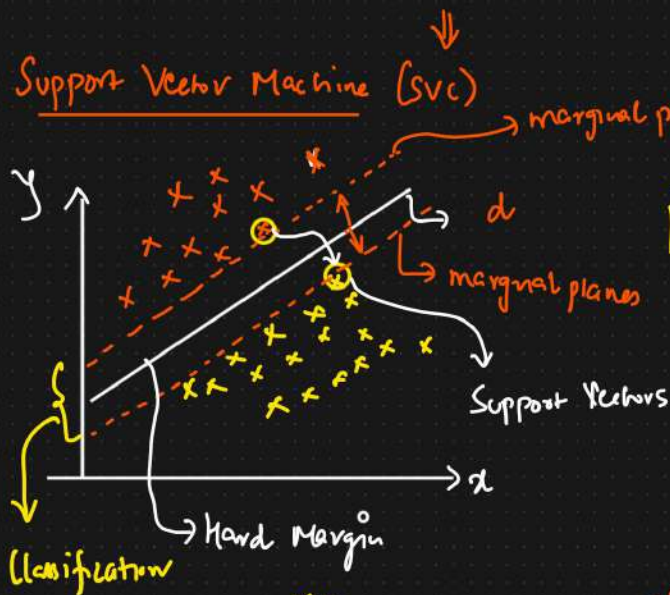
Support Vector Machines ML Algorithms.

① SVC (Support Vector Classifier)

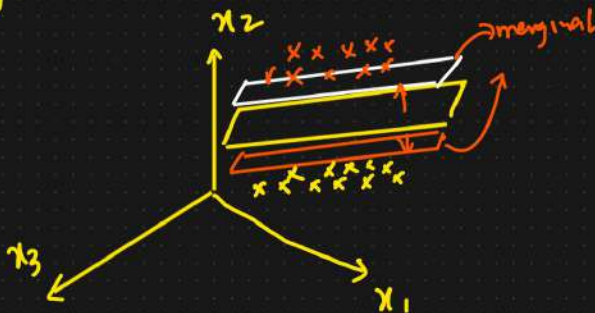
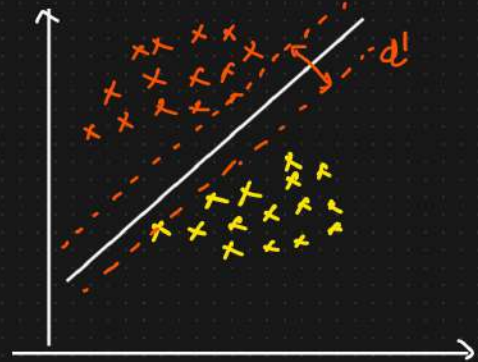
② SVR (Support Vector Regressor)



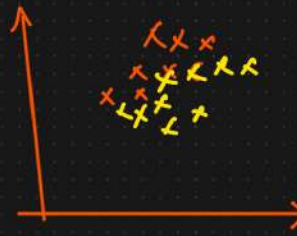
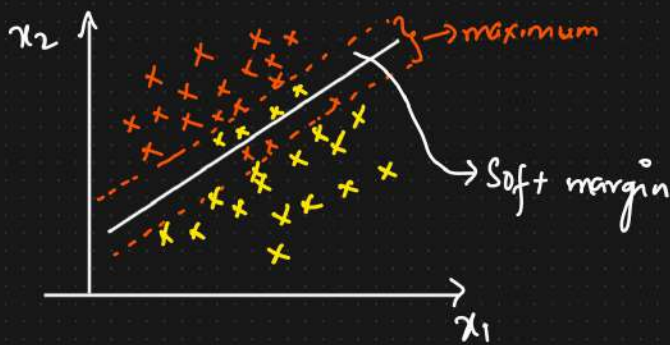
① Support Vector Machine (SVC)



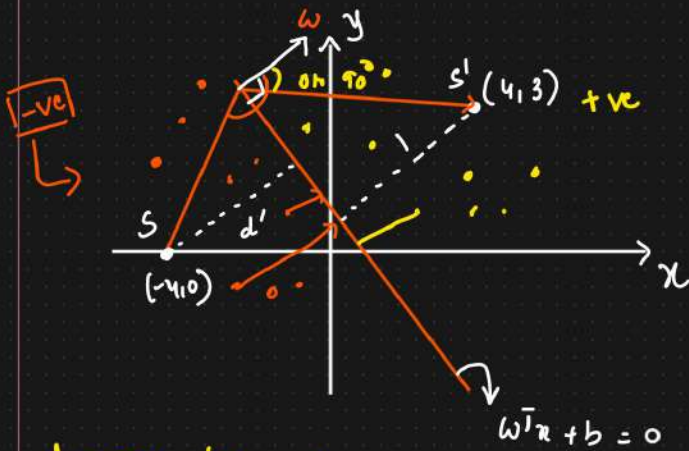
$$|d > d'|$$



Soft Margin And Hard Margin In SVM



Support Vector Machines (SVC) Maths Intuition



$$ax+by+c=0$$

↓

$$w_1x_1+w_2x_2+b=0$$

$$\boxed{w^T x + b = 0}$$

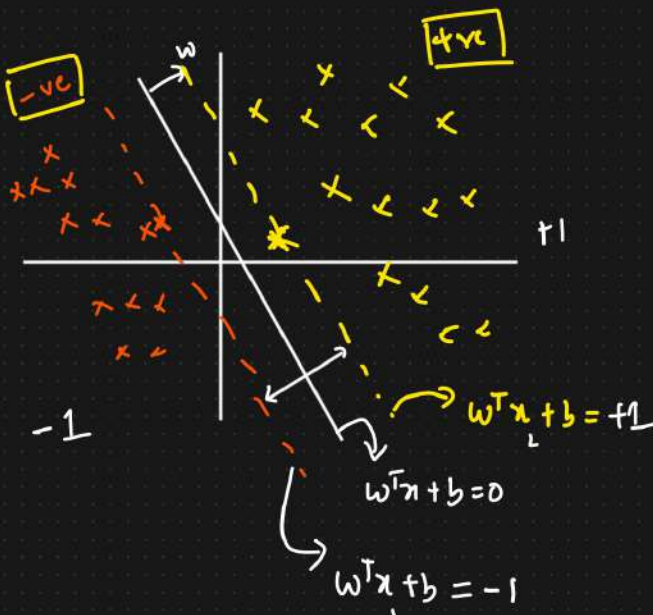
↓

$$b=0$$

$$\boxed{w^T x = 0}$$

$d = -ve$ below plane

$d = +ve$ above plane



$$w^T x_1 + b = 1$$

$$w^T x_2 + b = -1$$

$$\begin{matrix} (-) & (-) & (+) \end{matrix}$$

$$\frac{w^T (x_1 - x_2)}{\|w\|} = \frac{+2}{\|w\|}$$

Unit vector { Magnitude of the vector is 1 }

Cost function

Maximize $\frac{2}{\|w\|}$ $\Rightarrow$ Distance between Marginal plane
 w, b

Constraint such that

$$y_i \begin{cases} +1 & w^T x + b \geq 1 \\ -1 & w^T x + b \leq -1 \end{cases}$$

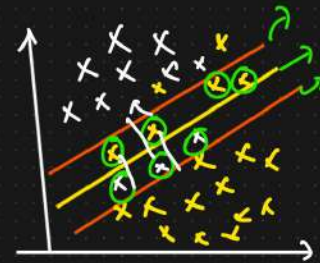
For all correct points

Constraint $\rightarrow y_i * (w^T x + b) \geq 1$

Maximize $\frac{2}{\|w\|}$
 w, b

$\Rightarrow \min_{(w,b)} \frac{\|w\|}{2}$

$C_i = 6$ ✓



Cost function of SVM (svc)

Min $\frac{\|w\|}{2}$
 w, b

$+ \sum_{i=1}^n C_i \xi_i$

Hinge Loss

$\Downarrow$

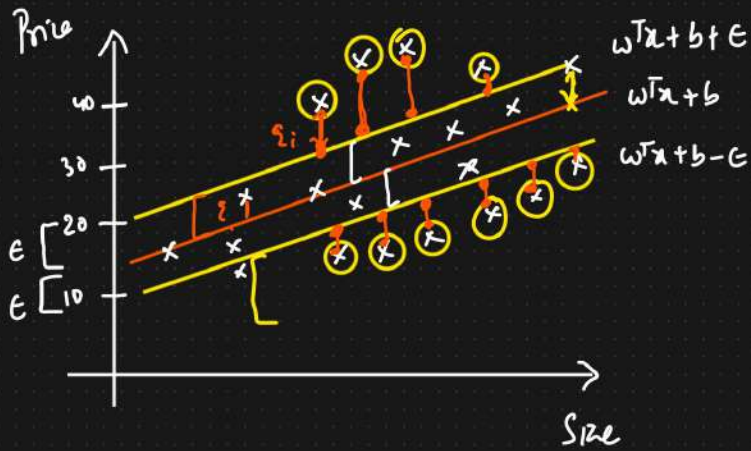
Soft margin

{ how many points we want to avoid misclassification }

{ distance of the incorrect data points from the marginal plane }

Support Vector Regression

ϵ : Marginal Error



Cost function

$$\text{Min}_{w, b} \frac{\|w\|^2}{2} + \underbrace{\left[C \sum_{i=1}^n \xi_i \right]}_{\text{hyperparameter}} \rightarrow \text{Hinge Loss}$$

Constraint :

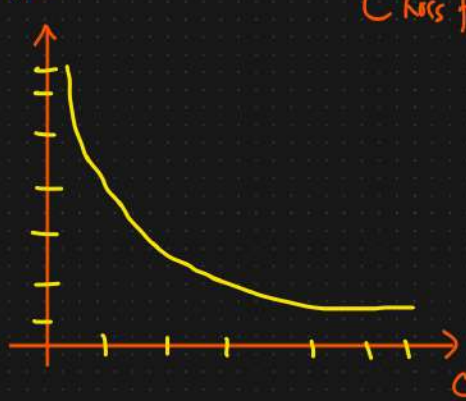
$$|y_i - w_i x_i| \leq \epsilon + \xi_i$$

$\Downarrow$
loss function

$\epsilon \rightarrow$ margin error

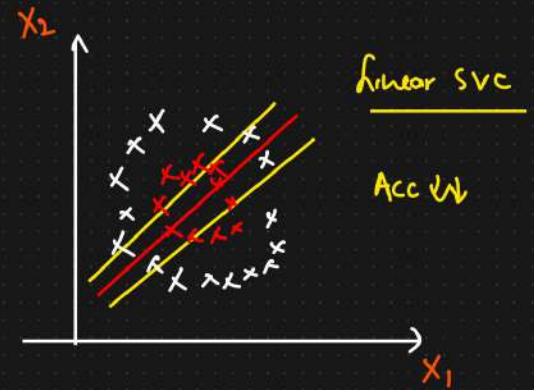
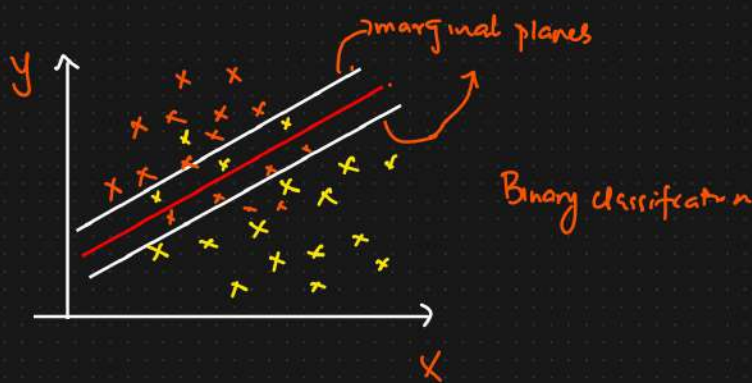
$\xi_i \rightarrow$ Error above the margin

loss function



Relationship
 $\left\{ \begin{array}{l} C \uparrow \uparrow \\ \text{loss function} \downarrow \end{array} \right.$

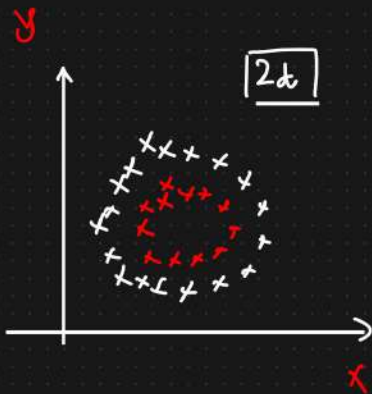
SVM KERNELS



Linear SVC



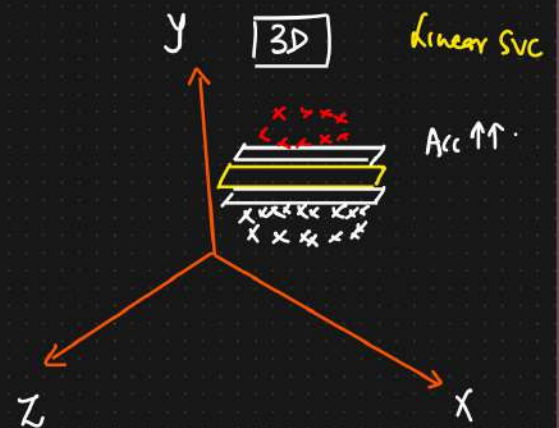
SVM Kernels



⇒ Transformations ⇒



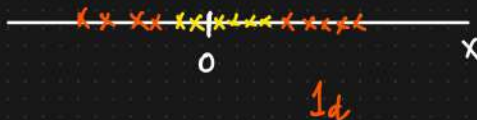
Mathematical
formulas



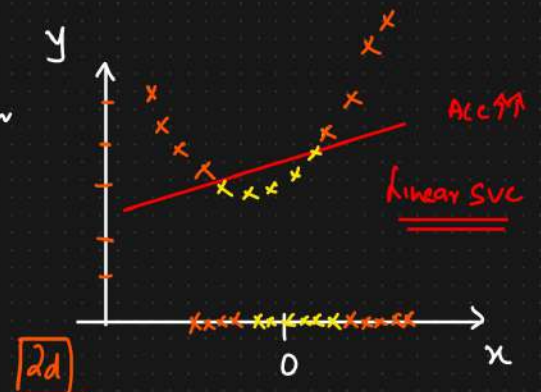
Dataset : 1d

SVM Kernel

Svc Linear SVC



⇒ Transformation
 $y = x^2$



① Polynomial Kernel

② RBF Kernel

③ Sigmoid Kernel

Naive Bayes' Algorithm (Classification)

① Probability

② Bayes' Theorem

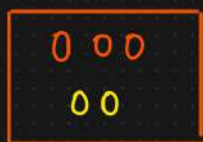
Independent Events

Rolling a Dice $\{1, 2, 3, 4, 5, 6\}$

$$Pr(1) = \frac{1}{6} \quad Pr(2) = \frac{1}{6} \quad Pr(3) = \frac{1}{6}$$

Dependent Events

① What is the probability of removing
a orange marble and then a yellow marble



$\hookrightarrow P(o) = \frac{3}{5} \rightarrow 1^{st} \text{ Event} \rightarrow \text{Orange marble has been removed}$



$\rightarrow P(y/o) = \frac{2}{4} = \frac{1}{2} \Rightarrow 2^{nd} \text{ Event} \Rightarrow \text{Removed the Yellow Marble}$

$$P(o \text{ and } y) = P(o) * \overset{\downarrow 1^{st}}{P(y/o)} \rightarrow \text{Conditional Probability}$$

$$= \frac{3}{5} * \frac{1}{2} = \boxed{\frac{3}{10}}$$

$$P(A \text{ and } B) = P(A) * P(B/A)$$

Bayes' Theorem

$$P(A \text{ and } B) = P(B \text{ and } A)$$

$$P(A) * P(B/A) = P(B) * P(A/B)$$

$$P(A/B) = \frac{P(A) * P(B/A)}{P(B)} \Rightarrow \text{Bayes' Theorem.}$$

$P(A/B)$ = Probability of Event A given B has occurred

$P(A)$ = Probability of Event A

$P(B)$ = Probability of Event B

$P(B/A)$ = Probability of Event B given A has occurred.

$$P(A/B) = \frac{P(A) * P(B/A)}{P(B)} \Rightarrow \text{Bayes' Theorem.}$$

I/P features			↓ Dependent
x_1	x_2	x_3	y
—	—	—	Yes
—	—	—	No
—	—	—	Yes
—	—	—	No

$$P(y/(x_1, x_2, x_3)) = \frac{P(y) * P(x_1, x_2, x_3|y)}{P(x_1, x_2, x_3)}$$

$$P(y/(x_1, x_2, x_3)) = \frac{P(y) * P(x_1, x_2, x_3)/y}{P(x_1, x_2, x_3)}$$

$$= \frac{P(y) * P(x_1/y) * P(x_2/y) * P(x_3/y)}{P(x_1) * P(x_2) * P(x_3)}$$

I/P features			↓ Dependent
x_1	x_2	x_3	y
-	-	-	Yes
-	-	-	No
-	-	-	Yes
-	-	-	No

$[0.60, 0.40]$
 $\Downarrow$
 Yes

New test data

$$Pr(y_{us}/(x_1, x_2, x_3)) = \frac{P(y_{us}) * P(x_1/y_{us}) * P(x_2/y_{us}) * P(x_3/y_{us})}{\cancel{P(x_1) * P(x_2) * P(x_3)}} \Rightarrow \text{Constant} = [0.60]$$

$\Downarrow$
Yes

$$Pr(N_0/(x_1, x_2, x_3)) = \frac{P(N_0) * P(x_1/N_0) * P(x_2/N_0) * P(x_3/N_0)}{\cancel{P(x_1) * P(x_2) * P(x_3)}} \Rightarrow \text{Constant} = 0.40$$

Let's Solve this Problem

Outlook

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

	Yes	No	$P(E/Yes)$	$P(E/No)$
Sunny	2	3	$\frac{2}{9}$	$\frac{3}{5}$
Overcast	4	0	$\frac{4}{9}$	$\frac{0}{5}$
Rain	3	2	$\frac{3}{9}$	$\frac{2}{5}$

Temperature $\rightarrow$ Test (Sunny, Hot) $\rightarrow$ O/P PLAY (Y/N)

	Yes	No	$P(E/Yes)$	$P(E/No)$		$P(Yes)$	$P(No)$
Hot	2	2	$\frac{2}{9}$	$\frac{2}{5}$	Yes	9	$\frac{9}{14}$
Mild	4	2	$\frac{4}{9}$	$\frac{4}{5}$	No	5	$\frac{5}{14}$
Cool	3	1	$\frac{3}{9}$	$\frac{3}{5}$			

$$P(Yes/Sunny, Hot) = \frac{P(Yes) * Pr(Sunny/Yes) * Pr(Hot/Yes)}{\cancel{Pr(Sunny)} * \cancel{Pr(Hot)}}$$

$$= \frac{9}{14} * \frac{2}{9} * \frac{2}{5}$$

$$= \frac{2}{63} = 0.031$$

$$P(No/(Sunny, Hot)) = P(No) * Pr(Sunny/No) * Pr(Hot/No)$$

$$= \frac{5}{14} * \frac{3}{5} * \frac{2}{5}$$

$$= \frac{3}{35} = 0.085$$

$$Pr(Yes/(Sunny, Hot)) = \frac{0.031}{(0.031 + 0.085)} = 0.27 = 27\%$$

$$P(\text{No} | \text{Sunny, hot}) = \frac{0.085}{(0.031 + 0.085)} = 0.73 = 73\%$$

New Test $\Rightarrow$ Outlook Sunny Temperature Hot O/p 73% $\Rightarrow$ They will not play Tennis

27% $\Rightarrow$ They will play Tennis.



0 $\Rightarrow$ Person is not ^{going to} playing Tennis

Variants of Naive Bayes

- ① Bernoulli Naive Bayes
- ② Multinomial Naive Bayes
- ③ Gaussian Naive Bayes

① Bernoulli Naive Bayes

Whenever your features are following a Bernoulli Distribution, then we need to use Bernoulli Naive Bayes Algorithm.

Dataset

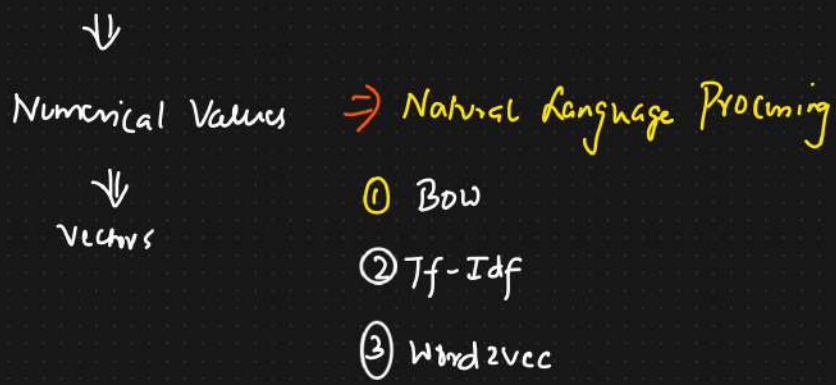
Bernoulli $\rightarrow 0, 1$

f_1	f_2	f_3	O/P
Yes	Pass	Male	Yes
Yes	Fail	Female	No
No	Pass	Male	Yes
Yes	Fail	Female	No

② Multinomial Naive Bayes $\Rightarrow$ I/p = Text

Dataset : Spam Classification

I/p $\rightarrow$ Email Body feature	O/p Spam/Not Spam
You have Million \$\$ lottery	Spam
KRISH YOU HAVE DONE GOOD JOB	HAM

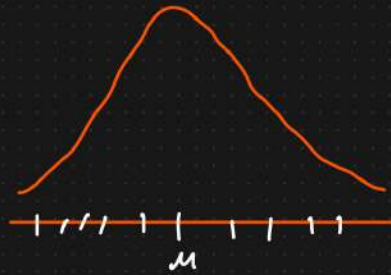


③ Gaussian Naive Bayes

If the features are following Gaussian Distribution, then we use
Gaussian Naive Bayes

DATASET → CONTINUOUS

IRIS Dataset



Age	Height	Weight	Yes/No
25	170	78	
38	160	75	
22	150	60	
29	170	35	

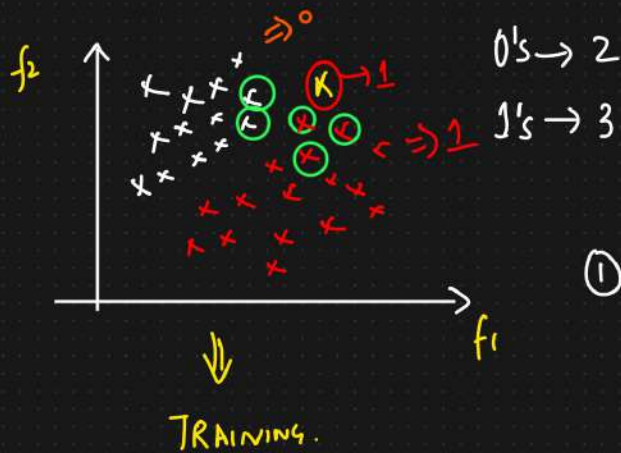
K Nearest Neighbour (KNN)

① Classification

② Regression

① Classification

K=5



f_1	f_2	y [Binary categories]
-	-	0
		1

① We have to initialize the K value

K=5

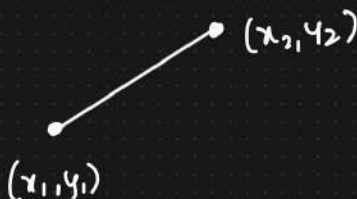
$K > 0 \dots \infty$

$K=1, 2, 3, 4, 5 \dots \Rightarrow$ Hyperparameter

② Find the K Nearest Neighbour for The Test Data.

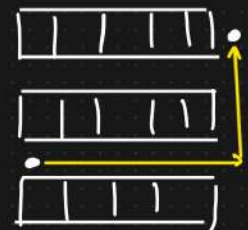
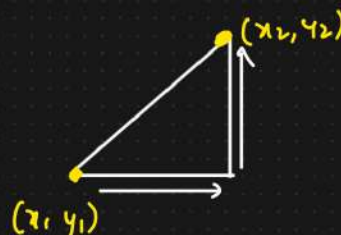
③ From those $K=5$ how many neighbour belong to 0 Category and 1 Category

① Euclidean Distance



$$\text{Distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

② Manhattan Distance



② Regression

House
price



Median

⇒ Average of all these
points to find out the
o/p,

Size

Time complexity

$O(n)$ ⇒ Million of data point

{ ① KD Tree
② Ball Tree } ⇒ Optimize

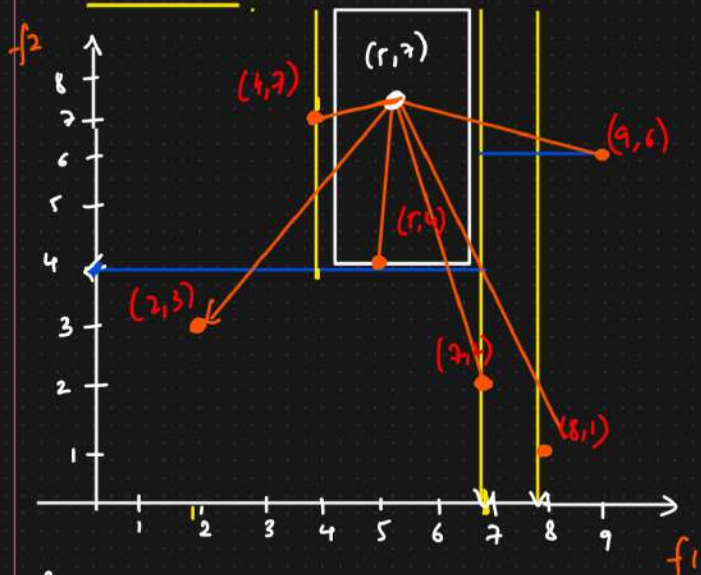


Binary Tree



Variants of KNN

KD Tree



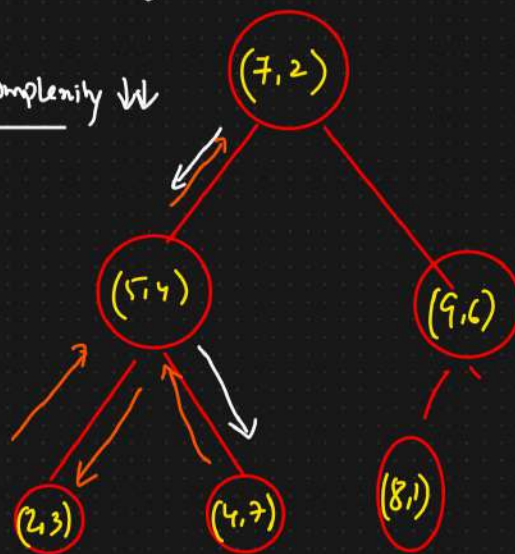
f_1
 2, 4, 5, 7 | 8, 9
 $\frac{5+7}{2} = \frac{12}{2} = 6.5$

f_2
 1, 2, 3, 4 | 6, 7

f_1	f_2
7	2
5	4
9	6
2	3
4	7
8	1
Median	Median

Binary Tree [KD Tree]

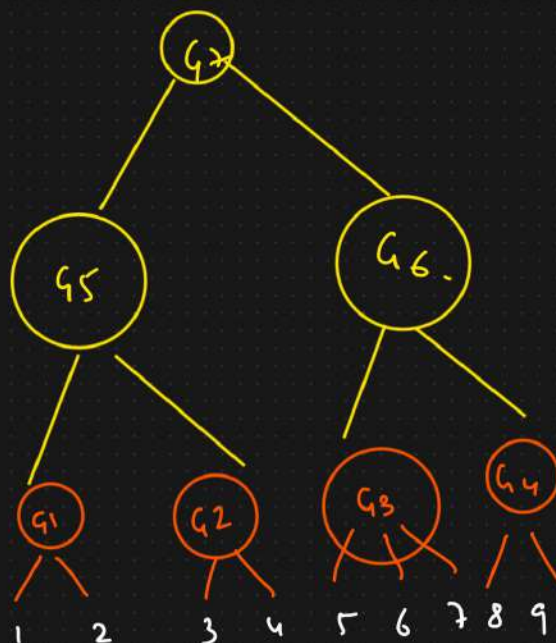
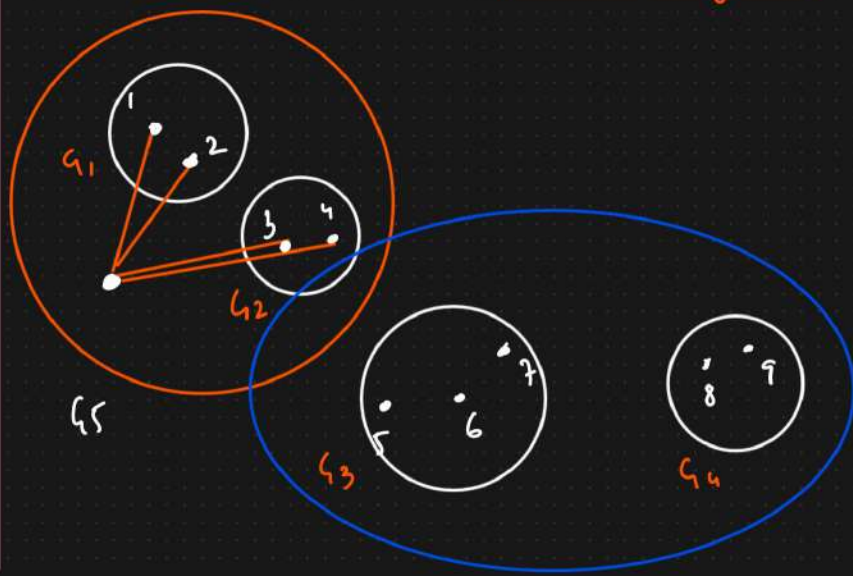
Time Complexity ↓



Back Tracking

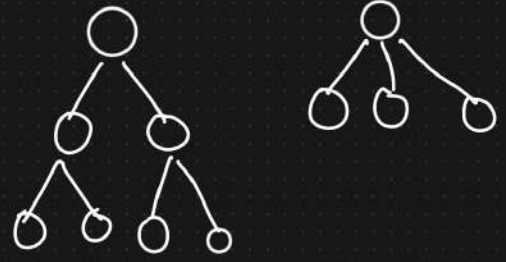
(2) Ball Tree

Time Complexity ↓



Decision Tree Classifier

Decision Tree Classifier $\rightarrow$ ID3
 $\rightarrow$ CART $\checkmark$



a) Entropy and Gini Index $\rightarrow$ Purity Split

b) Information Gain $\rightarrow$ features to select for
DT construction

age = 14

if (age ≤ 15):

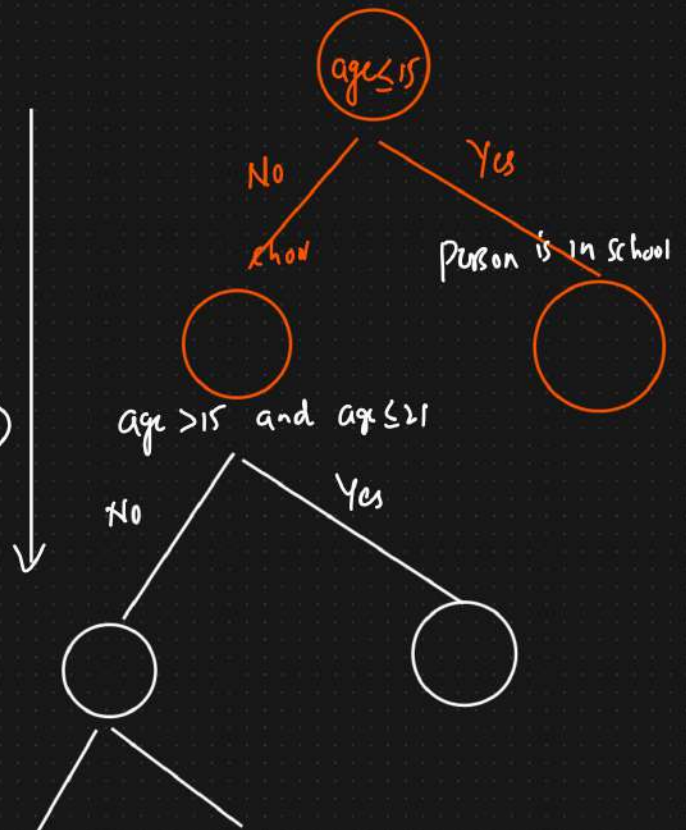
Print ("The person is in School")

elif (age > 15 and age ≤ 21):

Print ("The person may be college")

else:

Print ("The person has passed")



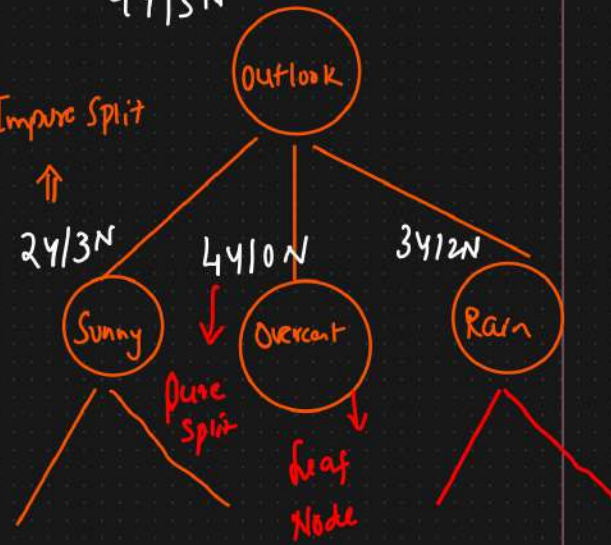
Dataset

Binary Classification

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Impure Split

94/5N



① Purity → Pure or Impure Split

↳ Entropy
↳ Gini Impurity

② What feature you need select for Splitting → Information Gain

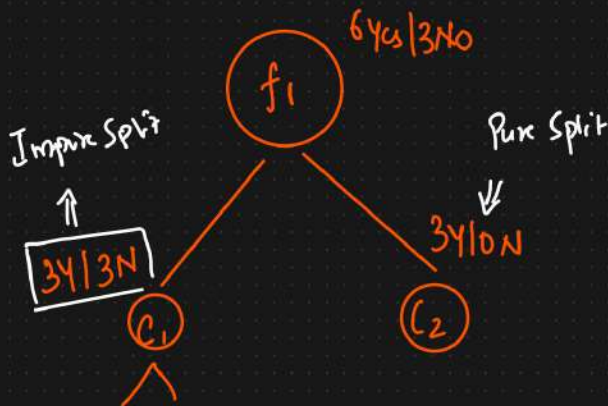
1
0
{ Binary Classification }

1) Entropy

$$H(S) = -P_+ \log_2 P_+ - P_- \log_2 P_-$$

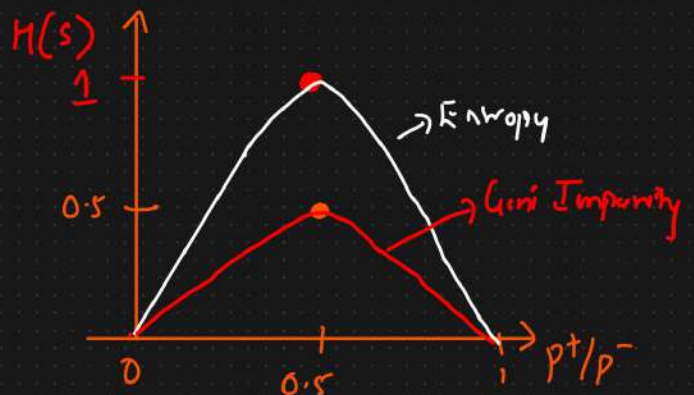
② Gini Impurity

$$G.I = 1 - \sum_{i=1}^n (P_i)^2$$



$$H(C_1) = -P_+ \log_2 P_+ - P_- \log_2 P_-$$

$$= -\frac{3}{6} \log_2 \frac{3}{6} - \frac{3}{6} \log_2 \frac{3}{6}$$



$$= 1 \Rightarrow \text{Impure Split}$$

$$H(c_2) = -\frac{3}{3} \log_2 \frac{3}{3} - 0 \log_2 0$$

$$= -1 \log_2 1 \Rightarrow 0 \Rightarrow \text{Pure Split}$$

② Gini Impurity

$$G.I = 1 - \sum_{i=1}^n (p_i)^2$$

$$= 1 - ((p_+)^2 + (p_-)^2)$$

$$= 1 - \left(\left(\frac{1}{2}\right)^2 + \left(\frac{1}{2}\right)^2\right)$$

$$= \underline{\underline{0.5}} \Rightarrow \text{Impure Split}$$

34/10N

$$= 1 - \left(\left(\frac{3}{3}\right)^2\right)$$

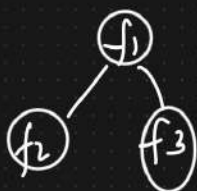
$$= 1 - 1$$

$$= \underline{\underline{0}} \Rightarrow \text{Pure Split}$$

$f_1 \quad f_2 \quad f_3$

Decision Tree

Split



$\Downarrow$
 $\Rightarrow$ Information Gain

Information Gain

$f_1 \quad f_2 \quad f_3 \quad O/P$

$$\text{Gain}(S, f_1) = \boxed{H(S)} - \sum_{v \in \text{val}} \frac{|S_v|}{|S|} H(S_v)$$

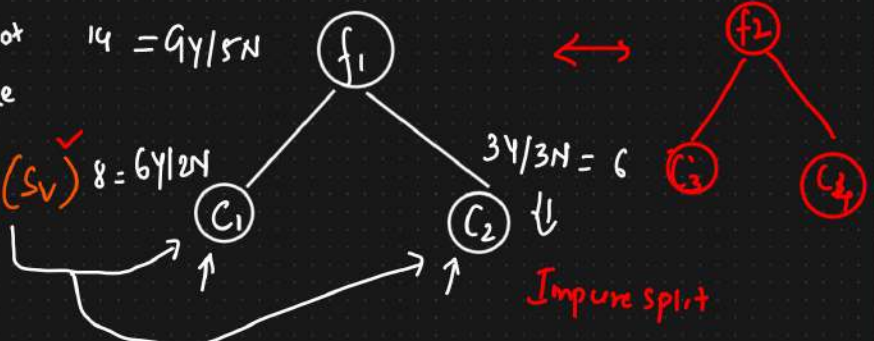
$\nearrow$ Entropy of the root node

$\nearrow$ Root Node

$$14 = 64/5N$$

$$8 = 64/2N$$

$$34/3N = 6$$



Impure split

$$H(S) = -p_+ \log_2 p_+ - p_- \log_2 p_-$$

$$= -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14}$$

$$\approx 0.94$$

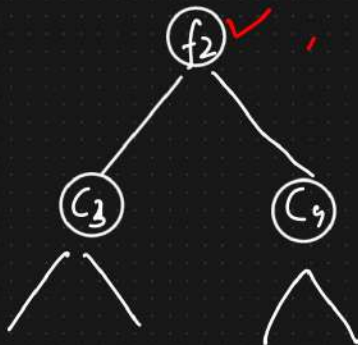
$$H(C_1) = -\frac{6}{8} \log_2 \frac{6}{8} - \frac{2}{8} \log_2 \frac{2}{8}$$

$$H(C_1) = 0.81$$

$$H(C_2) = 1$$

$$\text{Gain}(S, f_1) = 0.94 - \left[\frac{8}{14} \times 0.81 + \frac{6}{14} \times 1 \right]$$

$$\text{Gain}(S, f_1) = 0.049$$



$$\text{Gain}(S, f_2) = 0.051 > \text{Gain}(S, f_1) = 0.049$$

Information ^{Gain} is Basically calculated.

Entropy Vs Gini Impurity

$$H(S) = -p_+ \log_2 p_+ - p_- \log_2 p_-$$

$$G.I = 1 - \sum_{i=1}^n (p_i)^2 \Rightarrow$$

O/P = 3 categories

$$H(S) = -p_{C_1} \log_2 p_{C_1} - p_{C_2} \log_2 p_{C_2} - p_{C_3} \log_2 p_{C_3}$$

Whenever dataset is small $\rightarrow$ Entropy
 large $\rightarrow$ Gini Impurity

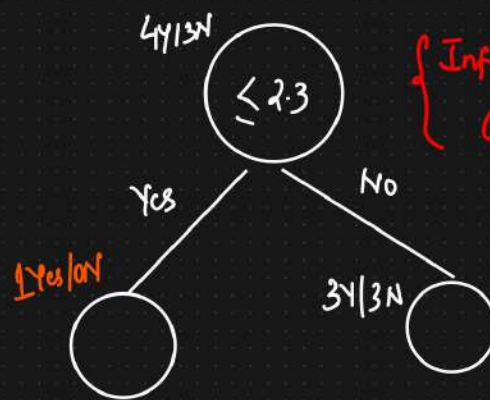
Decision Tree Split for Numerical Feature.

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

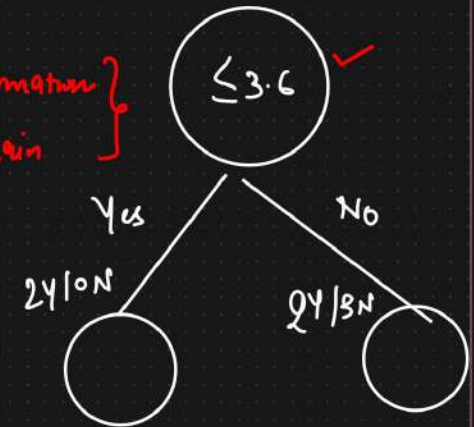
① Sort the feature value

f ₁	O/P
2.3	Yes
3.6	Yes
4	No
5.2	No
6.7	Yes
8.9	No
10.5	Yes

① Threshold = 2.3



② Threshold = 3.6

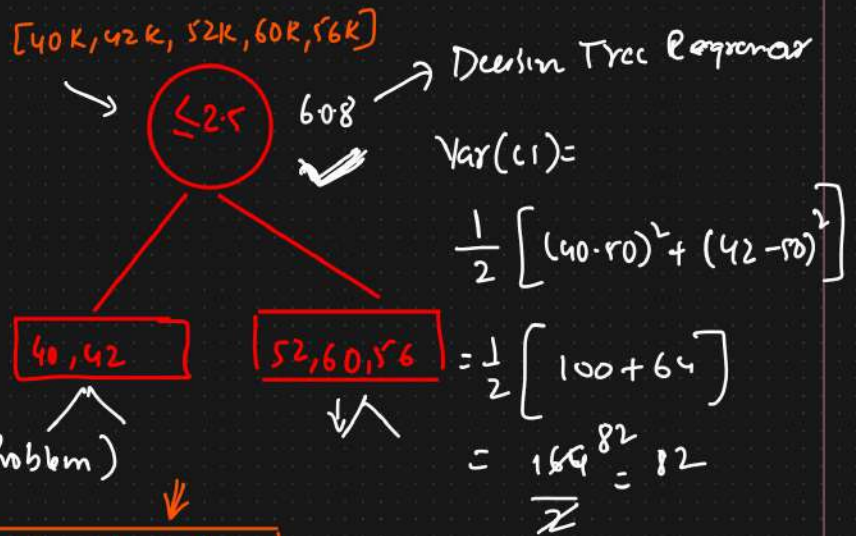
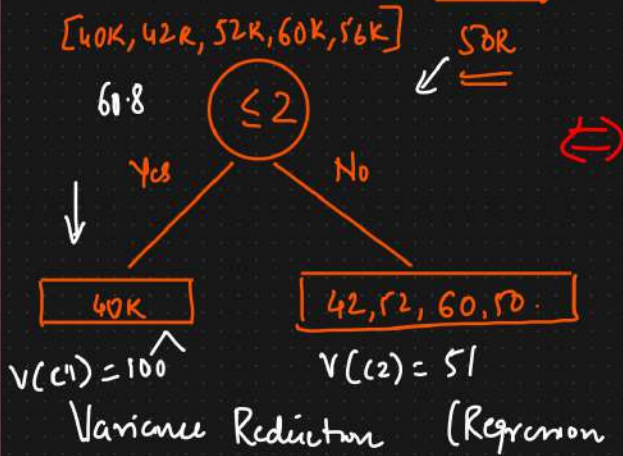
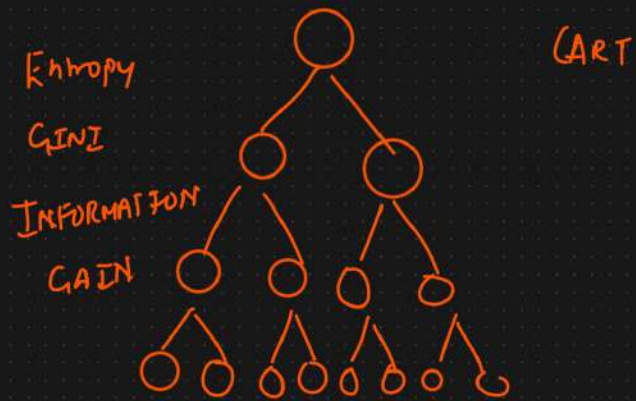


Millions of records

Time Complexity ↑↑

Decision Tree Regression

Exp	Career Gap	Salary
→ 2	Yes	40K
→ 2.5	Yes	42K
→ 3	No	52K
→ 4	No	60K
4.5	Yes	56K



① Variance = $\frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2$ → Average

Mean Square Error

Var(child2) = $\frac{1}{3} [4 + 100 + 36]$

$$= \frac{140}{3} = 46.66$$

Variance (Root) = $\frac{1}{5} [(40-50)^2 + (42-50)^2 + (52-50)^2 + (60-50)^2 + (56-50)^2]$

$$= \frac{1}{5} [100 + 64 + 4 + 100 + 36]$$

$$= 60.8$$

Variance Reduction

$$= 60.8 - \left[\frac{2}{5} \times 82 + \frac{3}{5} \times 46.66 \right]$$

$$= 60.8 - [32.8 + 27.996]$$

$$= +0.004$$

Variance (c1) = $\frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2$

$$= \frac{1}{1} (40-50)^2$$

$$= 100$$

$$\text{Variance (2)} = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2$$

$$= \frac{1}{4} \left[(42-50)^2 + (52-50)^2 + (60-50)^2 + (56-50)^2 \right]$$

$$= \frac{1}{4} [64 + 4 + 100 + 36]$$

$$= 51$$

Variance Reduction

$$\downarrow$$

$$= \text{Var}(\text{Root}) - \sum w_i \text{Var}(\text{child})$$

$$= 60.8 - \left[\frac{1}{5} \times 20 + \frac{4}{5} \times 51 \right]$$

$$= 60.8 - 20 - 40.8$$

Variance Reduction = 0

0 0.004

$$\text{Variance Reduction (Left Split)} < \text{VR (Right Split)}$$

Left Tree

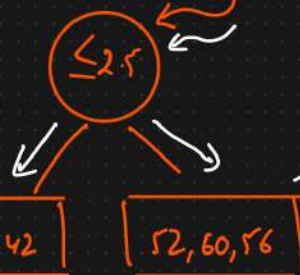


Leaf Node

$$\underline{\underline{\text{O/P}}} = \left[\frac{40+42}{2} \right]$$

$$= \underline{\underline{41}}$$

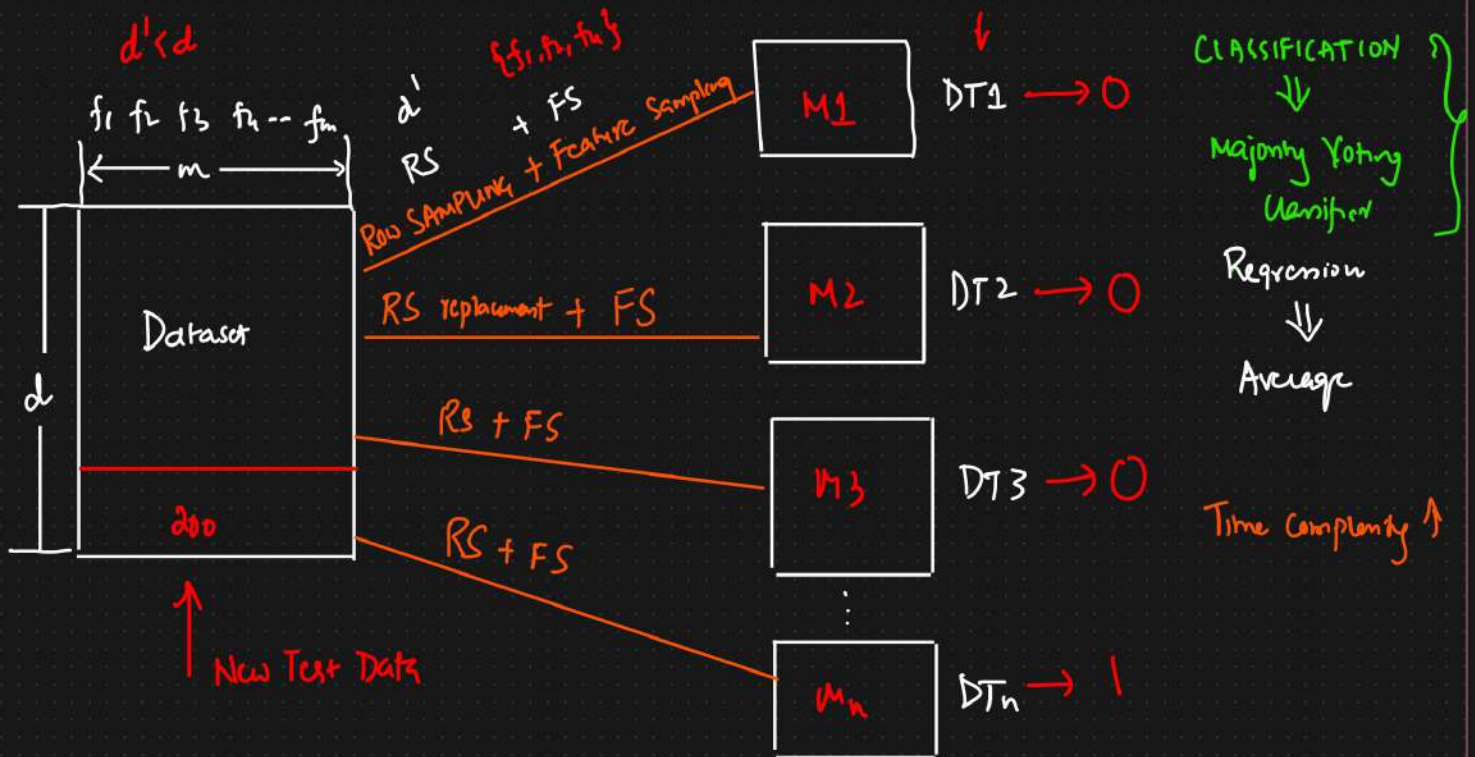
Test data



Decision Tree
O/P Req.

$$\frac{52+60+56}{3} = \underline{\underline{56}}$$

Random Forest CLASSIFICATION AND REGRESSIONS



CLASSIFICATION - MAJORITY VOTING CLASSIFIER

REGRESSION - Average O/P of the Models.

Why should we use Random Forest instead of DT?

Decision Tree

Generalized Model $\begin{cases} \rightarrow \text{Low Bias} \\ \rightarrow \text{Low Bias} \end{cases}$

Overfitting

Random Forest

Train Acc ↑ $\rightarrow$ Low Bias $\rightarrow$ Low Bias

Test Acc ↓ $\rightarrow$ High Variance $\rightarrow$ Low Variance

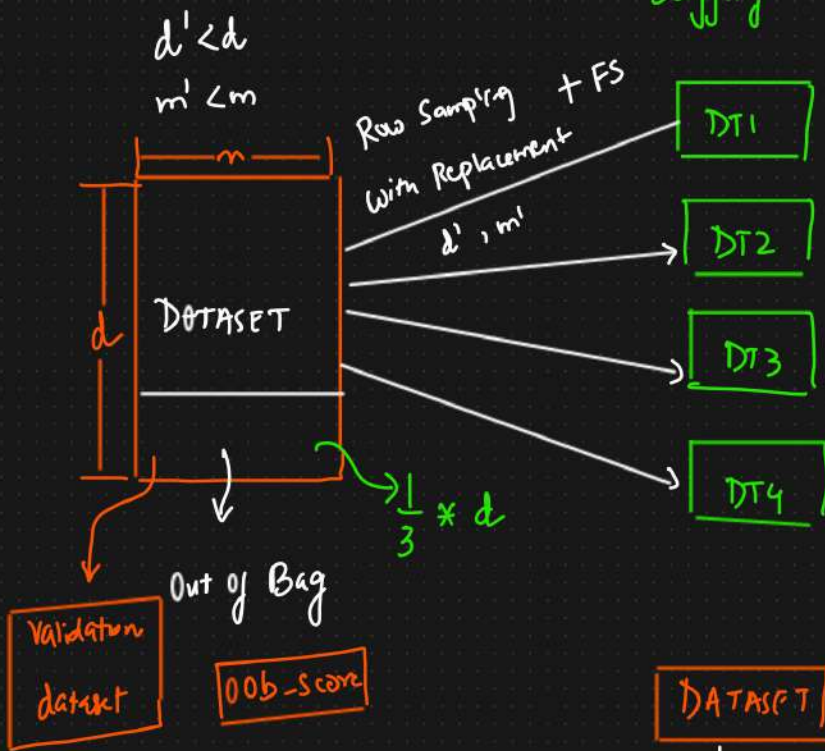


Out of Bag Evaluation (Random Forest)

Out of Bag score?



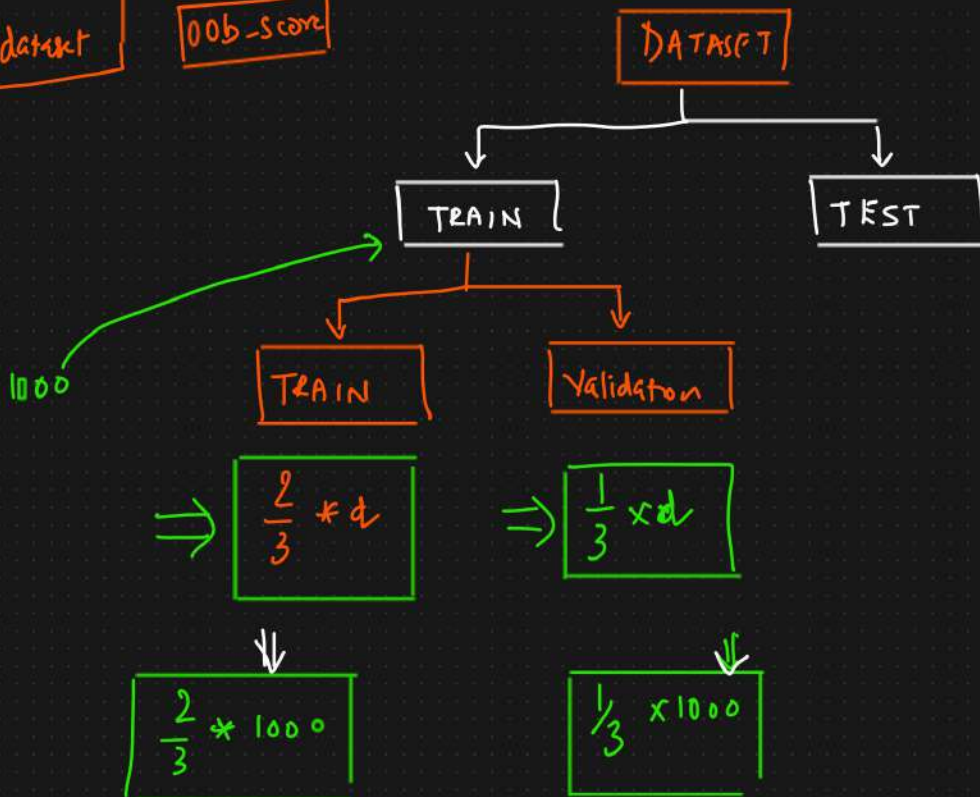
Bagging



Sklearn

Oob = TRUE

↓ ↓
[Oob Score] ⇒ Percentage
↓
Validation of Out of Bag DATA



AdaBoost Machine Learning Algorithms

Decision Trees

Ensemble Techniques

{Base Learners}

Bagging

- ① Random Forest Classifier
- ② Random Forest Regressor

{weak learners}

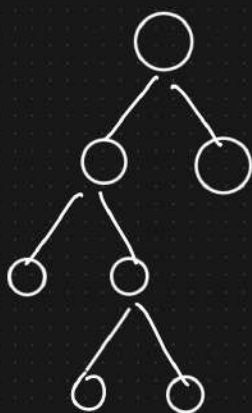
Boosting

- ① Adaboost
- ② Gradient Boosting
- ③ Xgboost

Decision Tree :

Overfitting : Train Acc ↑
Test Acc ↓

Low Bias
High Variance



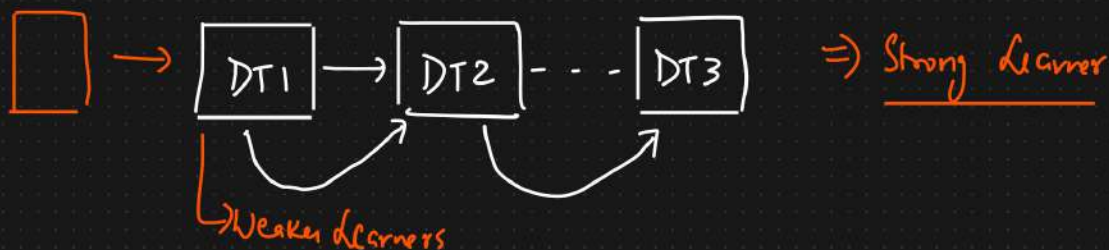
Random Forest

{Bagging}

Low Bias
Low Variance

Boosting

{Sequentially connected}



Weak Learner → Haven't learnt much from the
Training Dataset

Random Forest → Majority Voting classifier
Average of (O/P)

AdaBoost $\rightarrow$ Assignment weights to the weak learner

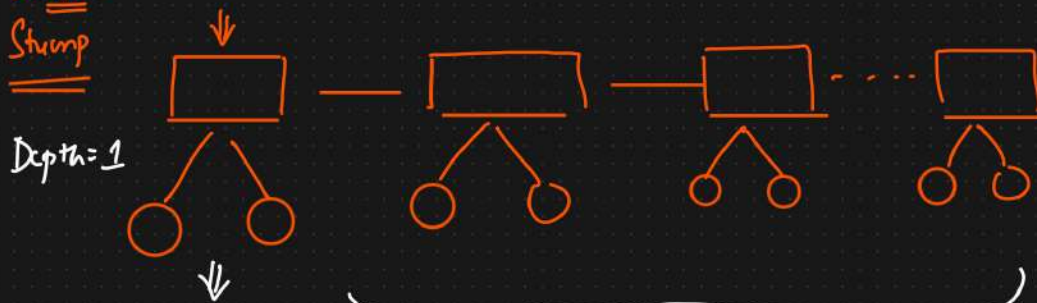
$M_1, \dots, M_n \rightarrow$ Decision Tree Stumps

$$f = \alpha_1(M_1) + \alpha_2(M_2) + \alpha_3(M_3) + \dots + \alpha_n(M_n)$$

$\{\alpha_1, \alpha_2, \alpha_3, \dots, \alpha_n\} \Rightarrow$ weights

$\rightarrow$ CLASSIFICATION
 $\rightarrow$ REGRESSION

Decision Stump



Weak learner

$\rightarrow$ Underfitting

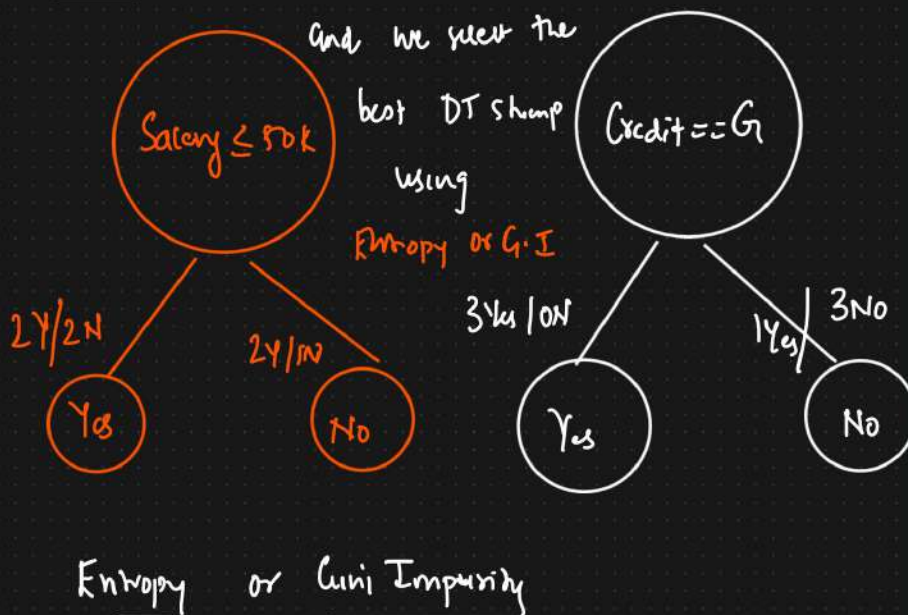
$\rightarrow$ Train Acc $\downarrow$ 40%
 $\rightarrow$ Test Acc $\uparrow$ 45%

Decision Tree Stump

$\left\{ \begin{array}{l} \text{High Bias} \\ \text{Low Variance} \end{array} \right\} \Rightarrow \left\{ \begin{array}{l} \text{Low Bias} \\ \text{High Variance} \end{array} \right\}$

Adaboost Classifier Maths Indepth Intuition ① We create Decision Tree Stump

Salary	Credit	Approval
$\leq 50K$	B	No
$\leq 50K$	G	Yes
$\leq 50K$	G	Yes
$> 50K$	B	No
$> 50K$	G	Yes
$> 50K$	N	Yes
$\leq 50K$	N	No

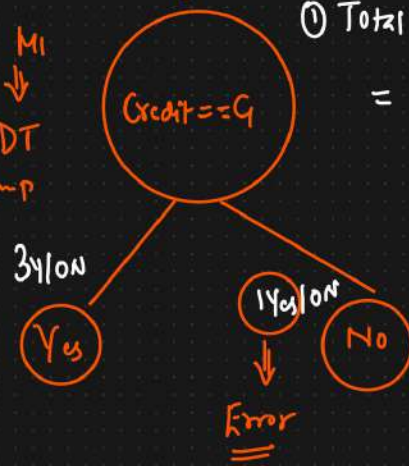


$$H(c) = -p_+ \log_2 p_+ - p_- \log_2 p_-$$

② Sum of the Total Errors And Performance of Stump

Salary	Credit	Approval	Sample Weights
$\leq 50K$	B	No	$\frac{1}{7}$
$\leq 50K$	G	Yes	$\frac{1}{7}$
$\leq 50K$	G	Yes	$\frac{1}{7}$
$> 50K$	B	No	$\frac{1}{7}$
$> 50K$	G	Yes	$\frac{1}{7}$
$> 50K$	N	Yes	$\frac{1}{7}$
$\leq 50K$	N	No	$\frac{1}{7}$

M_1
↓
or DT
Stump



Sum of all the
① Total Error
= $\frac{1}{7}$

② Performance of Stump

$$= \frac{1}{2} \ln \left[\frac{1 - TE}{TE} \right]$$

$$= \frac{1}{2} \ln \left[\frac{1 - \frac{1}{7}}{\frac{1}{7}} \right]$$

$$= \frac{1}{2} \ln [6] \approx \underline{\underline{0.896}}$$

Performance of Stump ≈ 0.896

$$f = d_1(M_1) + d_2(M_2) + \dots + d_n(M_n)$$

$d_1 = 0.896 \Rightarrow$ weight

③ Update the weights for correctly and Incorrectly classified points

Salary	Credit	Approval	Sample Weights	update wts
$\leq 50K$	B	No	$\frac{1}{7} \downarrow$	$\rightarrow 0.058$
$\leq 50K$	G	Yes	$\frac{1}{7} \downarrow$	$\rightarrow 0.058$
$\leq 50K$	G	Yes	$\frac{1}{7} \downarrow$	$\rightarrow 0.058$
$> 50K$	B	No	$\frac{1}{7} \downarrow$	$\rightarrow 0.058$
$> 50K$	G	Yes	$\frac{1}{7} \downarrow$	$\rightarrow 0.058$
$> 50K$	N	Yes	$\frac{1}{7} \uparrow$	$\rightarrow 0.349$
$\leq 50K$	N	No	$\frac{1}{7} \downarrow$	$\rightarrow 0.058$

For correct classified points

= weight * $e^{-\text{Performance of stump}}$

$$= \frac{1}{7} * e^{-(0.896)}$$

$$= \underline{\underline{0.058}}$$

For Incorrect classified

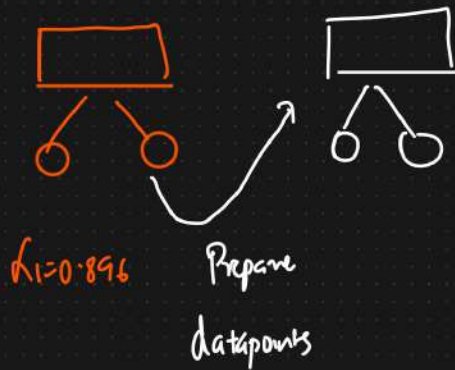
= weight * $e^{\text{Performance of stump}}$

$$= \frac{1}{7} * e^{(0.896)}$$

$$= 0.349$$

④ Normalized Weights Computation And Assigning Bins

Salary	Credit	Approval	update wts	Normalized Weights	Bins Assignment
$\leq 50K$	B	No	0.058	0.08	0 - 0.08
$\leq 50K$	G	Yes	0.058	0.08	0.08 - 0.16
$\leq 50K$	G	Yes	0.058	0.08	0.16 - 0.24
$> 50K$	B	No	0.058	0.08	0.24 - 0.32
$> 50K$	G	Yes	0.058	0.08	0.32 - 0.40
$> 50K$	N	Yes	0.349	0.50	0.40 - 0.70
$\leq 50K$	N	No	0.058	0.08	0.70 - 0.58
			0.697	≈ 1	



$$R_1 = 0.896$$

⑤ Select data points to send to Next Shump

① Iteration process selecting random value between 0 and 1

Salary	Credit	Approval	Bins Assignment
$\leq 50K$	B	No	0 - 0.08
$\leq 50K$	G	Yes	0.08 - 0.16
$\leq 50K$	G	Yes	0.16 - 0.24
$> 50K$	B	No	0.24 - 0.32
$> 50K$	G	Yes	0.32 - 0.40
$> 50K$	N	Yes	0.40 - 0.70
$\leq 50K$	N	No	0.70 - 0.58

S	Credit	Approval	Random
$> 50K$	N	Yes	0.50
$\leq 50K$	G	Yes	0.10
$> 50K$	N	Yes	0.60
$> 50K$	N	Yes	0.75
$\leq 50K$	G	Yes	0.24
$> 50K$	B	No	0.32
$> 50K$	N	Yes	0.87

⑥ These records will be sent to Next DT Stump.

S	Credit	Approval	Sample weight
$>50K$	N	Yes	$\frac{1}{6}$
$\leq 50K$	G	Yes	$\frac{1}{6}$
$>50K$	N	Yes	$\frac{1}{6}$
$>50K$	N	Yes	$\frac{1}{6}$
$\leq 50K$	G	Yes	$\frac{1}{6}$
$>50K$	B	No	$\frac{1}{6}$
$>50K$	N	Yes	$\frac{1}{6}$

TK

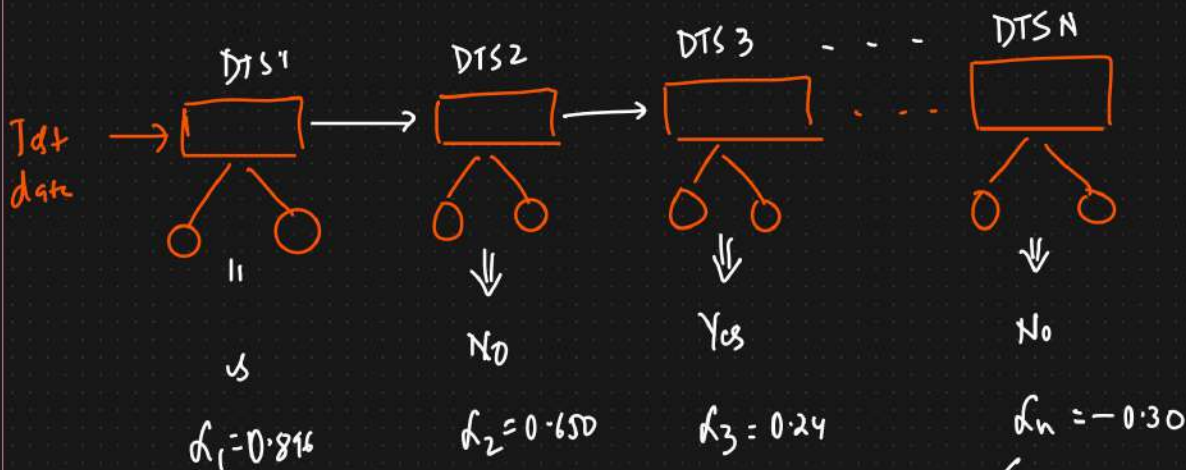
Performance Stump $\Rightarrow 0.65$

$$f_1 = \alpha_1(M_1) + \alpha_2(M_2)$$

$$\alpha_1 = 0.816 \quad \alpha_2 = 0.65$$

⑦ Final Prediction

Test data ($\leq 50K, G$)



$$f = \alpha_1(M_1) + \alpha_2(M_2) + \alpha_3(M_3) + \dots + \alpha_4(M_4)$$

$$= 0.816(\text{Yes}) + (0.650)(\text{No}) + 0.24(\text{Yes}) - 0.30(\text{No})$$

$$\begin{array}{r} 0.816 \\ 0.240 \\ \hline 1.056 \end{array}$$

$$= \boxed{1.136} (Y_u) + 0.350 (N_o) \quad \Rightarrow \underline{\underline{Op : Y_u}}$$

↑↑

↓↓

↑↑

→ Performance of say (Y_u) = 1.136

↓

Performance of say (N_o) = 0.350

GRADIENT BOOSTING ALGORITHM

① Regression

② CLASSIFICATION

Regression Dataset

Exp	Degree	Salary y	$\hat{y}$	$(y - \hat{y})$ R_1	R_2	$\hat{y}$	R_3	R_4
→ 2	B.E	50K	75K	-25K	-23	72.7	-22.7	-
3	Masters	70K	75K	-5K	-3	74.7	-4.7	-
5	Masters	80K	75K	5K	3	-	-	-
6	PHD	100K	75K	25K	20	-	-	-
		75K						

Steps

① Create a Base Model

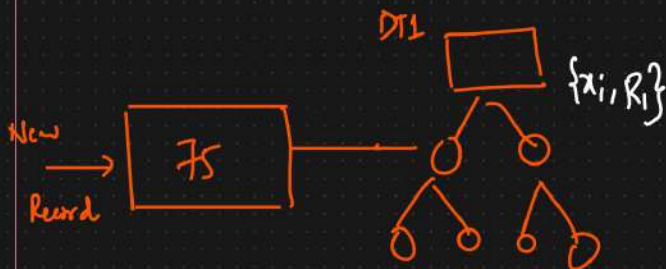
75

$$\text{Average} = \frac{50 + 70 + 80 + 100}{4} = 75$$

② Compute Residuals, Error

③ Construct a Decision Tree

Consider inputs x_i and o/p R_i



$$\text{Predicted o/p} \Rightarrow 75 + (-23) = 75 - 23 = \underline{\underline{52}} \text{ \{Overfitting\}}$$

Predicted o/p $\Rightarrow 75 + \alpha(DT_1) = 75 + (0.1)(-23)$

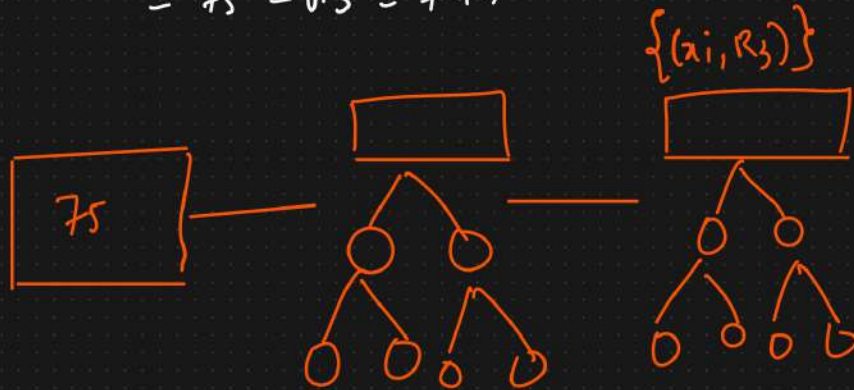
$\alpha = \text{learning rate}$
 $\alpha = 0.1$

$= 75 - 2.3$

$= 72.7$

$\Rightarrow 75 + 0.1(-3)$

$= 75 - 0.3 = 74.7$



$$F(x) = \alpha_0 h_0(x) + \alpha_1(h(x)) + \alpha_2(h(x)) + \alpha_3(h(x)) + \dots + \alpha_n(h(x))$$

Learning Rate $\alpha = 0.1$

$$F(x) = \sum_{i=0}^n \alpha_i h_i(x)$$

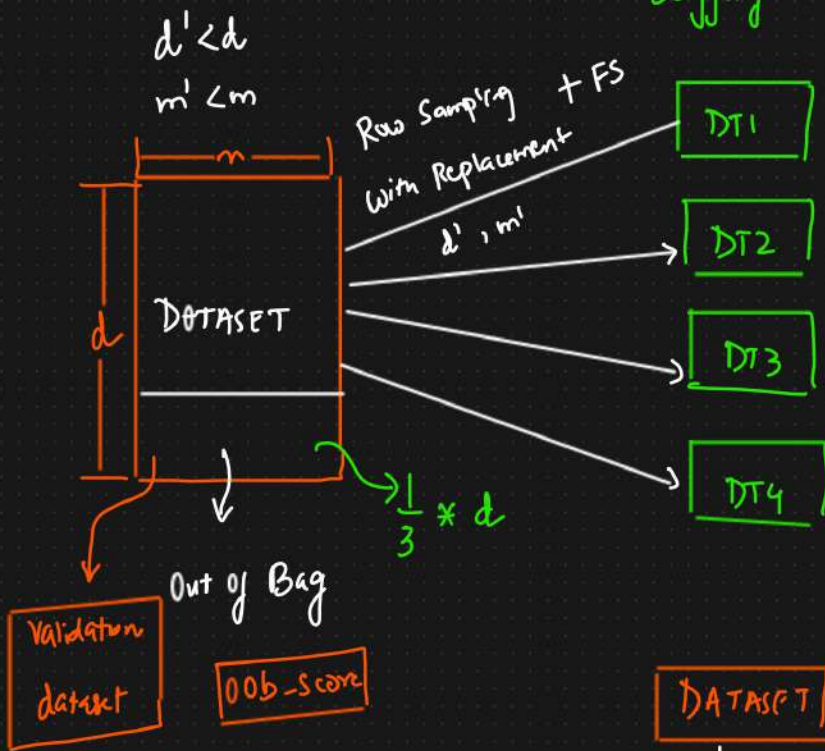
$\Rightarrow$ Final Function
 of Gradient
 Boosting

Out of Bag Evaluation (Random Forest)

Out of Bag score?



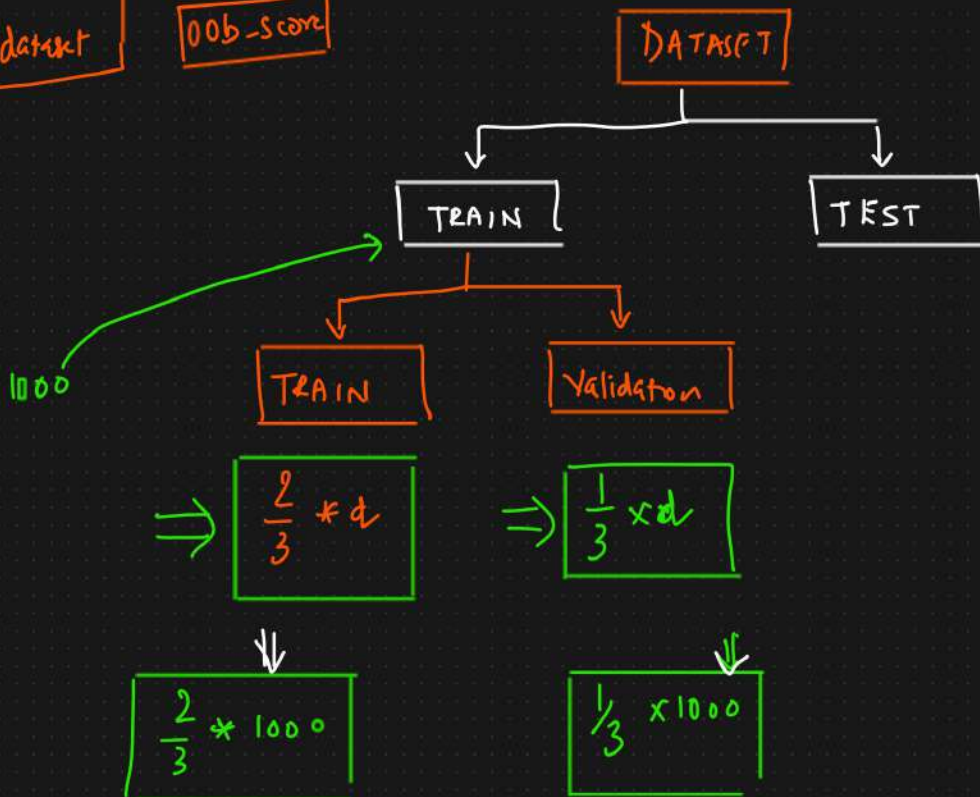
Bagging



SKlearn

`Oob = TRUE`

↓ ↓
[OOB Score] ⇒ Percentage
↓
Validation of Out of Bag DATA



Xgboost ML Algorithm (Classification)

$$\hat{y} = 0.5$$

Dataset

✓ Salary	✓ Credit	y	↓ R1	^ y	✓ R2
→ <=50K	B	0	-0.5	0.52	-0.48
<=50K	G	1	0.5	0.58	0.42
<=50K	G	1	0.5	-	-
>50K	B	0	-0.5	-	-
>50K	G	1	0.5	-	-
>50K	N	1	0.5	-	-
<=50K	N	0	-0.5	-	-

Steps

- ① Construct a base Model
- ② Construct a Decision Tree with root.

- ③ Calculate Similarity Weight

$$= \sum (\text{Residual})^2$$

$$\text{Odds value} \leftarrow \frac{\sum P_i (1 - P_i) + \lambda}{\lambda}$$

- ④ Calculate Gain

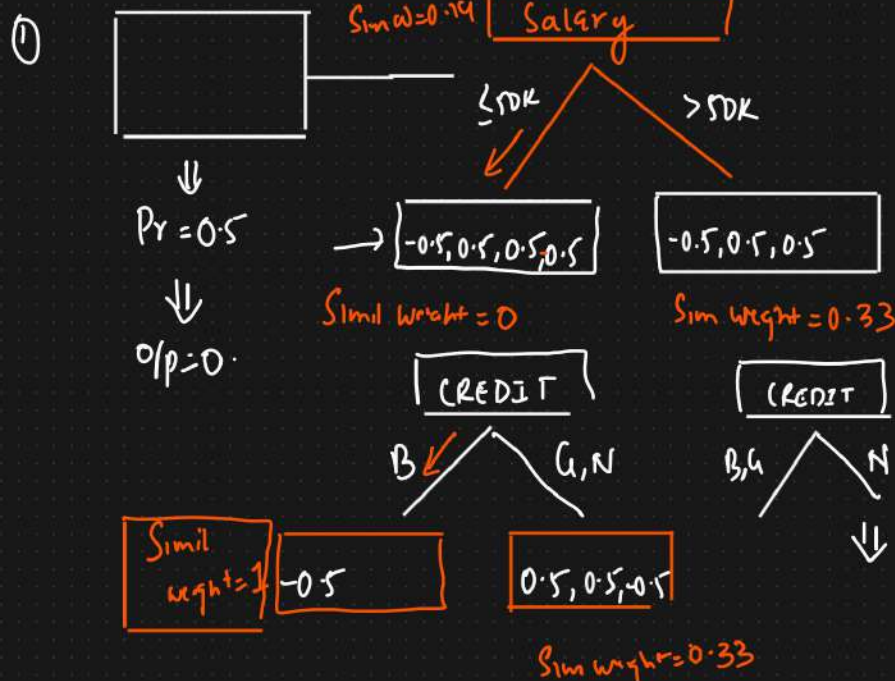
Hyper parameters

Test data

$$\log(\text{odds}) = \log\left(\frac{P}{1-P}\right)$$

$$\log(\text{odds}) = \log\left(\frac{0.5}{0.5}\right)$$

$$= 0$$



$$\text{Similarity}(dc) = \frac{0.25}{0.25} = 1$$

$$\text{Similarity}(Rc) = \frac{0.25}{0.75} = 0.33$$

$$\text{Gain} = 1 + 0.33 - 0 = \boxed{1.33}$$

$$\text{Similarity weight (dC)} = \frac{\sum (\text{Residual})^2}{\sum \text{Pr}(1-\text{Pr})}$$

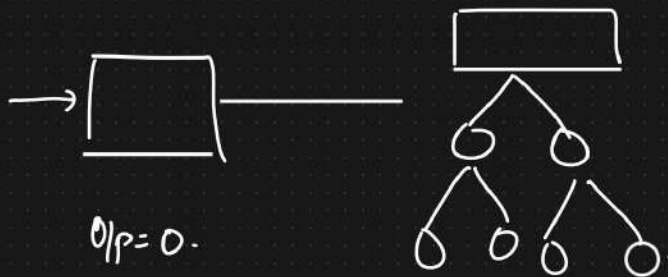
$$= \left[(-0.5 + 0.5 + 1.5 - 0.5)^2 \right] = 0$$

$$\text{Similarity weight (RL)} = \frac{(\cancel{1.5} + \cancel{0.5} + 0.5)^2}{0.75} = \frac{0.25}{0.75} = \frac{1}{3}$$

$$1 \leftarrow [0.5(1-0.5) + 0.5(1-0.5) + 1.5(1-0.5) + 0.5(1-0.5)]$$

$$\text{Gain} = 0 + 0.33 - 0.14 = 0.21$$

Final o/p



$$\text{New Date} \rightarrow = \sigma \left(0 + \overset{\text{Similarity weight}}{d(1)} \right) \quad d=0.1$$

Logistic function $\leftarrow$ Sigmoid Activation

$$= \sigma(0 + (0.1)(1))$$

$$= \frac{1}{1 + e^{-0.1}} = \underline{\underline{0.52}}$$

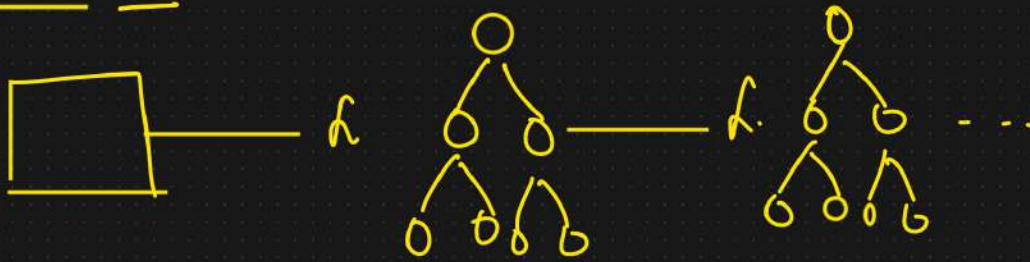
Second Record

$$\text{o/p} = \sigma(0 + d(0.33))$$

$$= \sigma(0 + 0.1(0.33))$$

$$= \frac{1}{1 + e^{-0.033}} = \underline{\underline{0.508}}$$

Xgboost classifier



$$O/p = \sigma \left(\text{Boosted model} + f_1(DT1) + f_2(DT2) + f_3(DT3) \right).$$

Xgboost Regressor Mh Algorithm



Exp	Gap	Salary	R_1	$\hat{y}$	R_2
→ 2	Yes	40K	-11	49.9	-9.9
→ 2.5	Yes	42K	-9	49.9	-7.9
→ 3	No	52K	1	51.5	0.5
4	No	60K	9	51.5	8.5
4.5	Yes	62K	11	52.1	9.9

$$[51 + (0.1)(-10)] = 51 - 0.1 = 49.9$$

$$[51 + (0.1)(5)] = 51.5$$

$$[51 + 0.1(11)] = 51 + 1.1 = 52.1$$

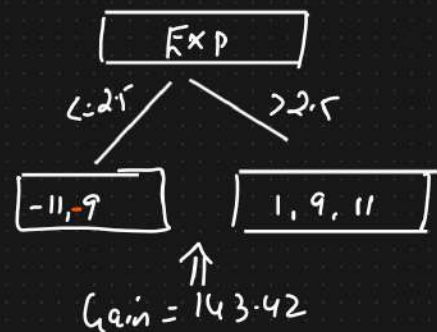
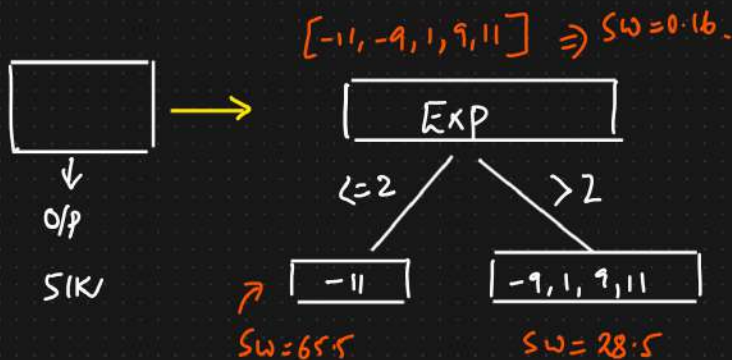
≈ 51K

$$\text{Similarity weight} = \frac{\sum (\text{Residual})^2}{\sum p_r(1-p_r)}$$

Gain

Steps

- 1) Create a Base Model
- 2) Residual Computation
- 3) Construct DT1 using $\{x_i, R_1\}$



$$\text{Similarity weight} = \frac{\sum (\text{Residual})^2}{\sum p_r(1-p_r)}$$

$$\lambda = 1$$

No. of Residuals + λ → Hyperparameter

$$SW(\text{left child}) = \frac{121}{1+1}$$

$$= 121/2 = 60.5$$

$$\lambda \uparrow \quad SW \downarrow$$

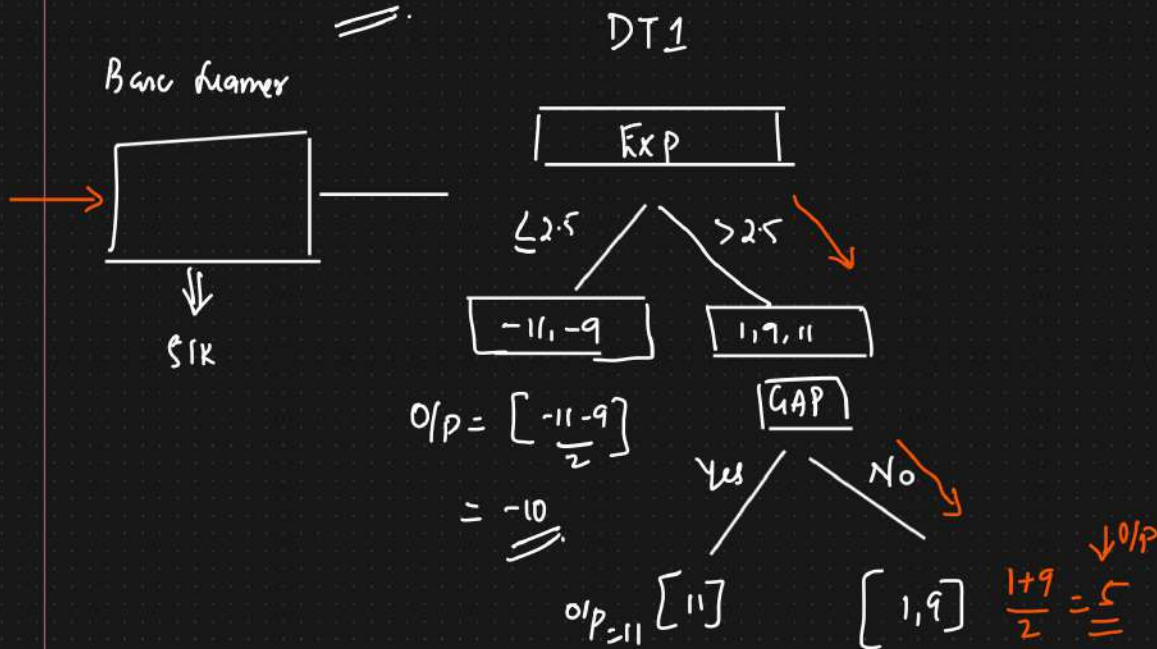
$$SW(\text{Right child}) = \frac{(-9+1+9+11)^2}{4+1}$$

$$= \frac{144}{5} = 28.5$$

⑤ Calculate Gain

$$\text{Gain} = 65.5 + 28.5 - 0.16$$

$$= 98.34$$



α = Learning Rate $\alpha = 0.1 \Rightarrow$ Hyperparameter

$$\text{XGBoost Classifier} = \text{Base learner} + \alpha_1(DT1) + \alpha_2(DT2) + \dots + \alpha_n(DTn)$$

$$\text{XGBoost Regressor} = \overset{\text{O/P}}{51K} + 0.1(5)$$

$$= 51 + 0.5$$

$$= \underline{\underline{51.5}}$$

$$\text{Similarity Weight} = \frac{\sum (\text{Residual})^2}{\text{No. of Residuals} + \boxed{\lambda}}$$

{ Regression }

Unsupervised Machine Learning

Supervised ML

- Regression
- Classification

I/p

	Dataset				
	f ₁	f ₂	f ₃	f ₄	Dependent or ↓ o/p features

Unsupervised ML $\rightarrow$ Clustering { group your data into similar clusters }

Datav t

Age	Experience	Salary
-----	------------	--------

No O/P

Chapter 9

A diagram of a cell in metaphase. The cell is represented by a circle. Inside, several X-shaped chromosomes are aligned in a row at the center of the cell, forming the equatorial plate. Spindle fibers are shown as thin lines extending from the top and bottom poles towards the centromeres of the chromosomes.

⇒ Unsupervised ML


$$\left\{ \begin{array}{ccc} - & - & - \\ - & - & - \end{array} \right.$$

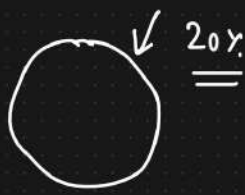
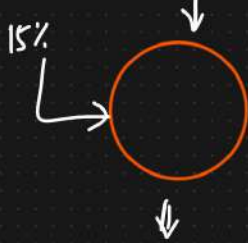
Group or cluster
this data

$$\left\{ \begin{array}{ccc} - & - & - \\ - & - & - \end{array} \right.$$

Eg: Customer Segmentation

Apple Product : Data Salary Spending-Score

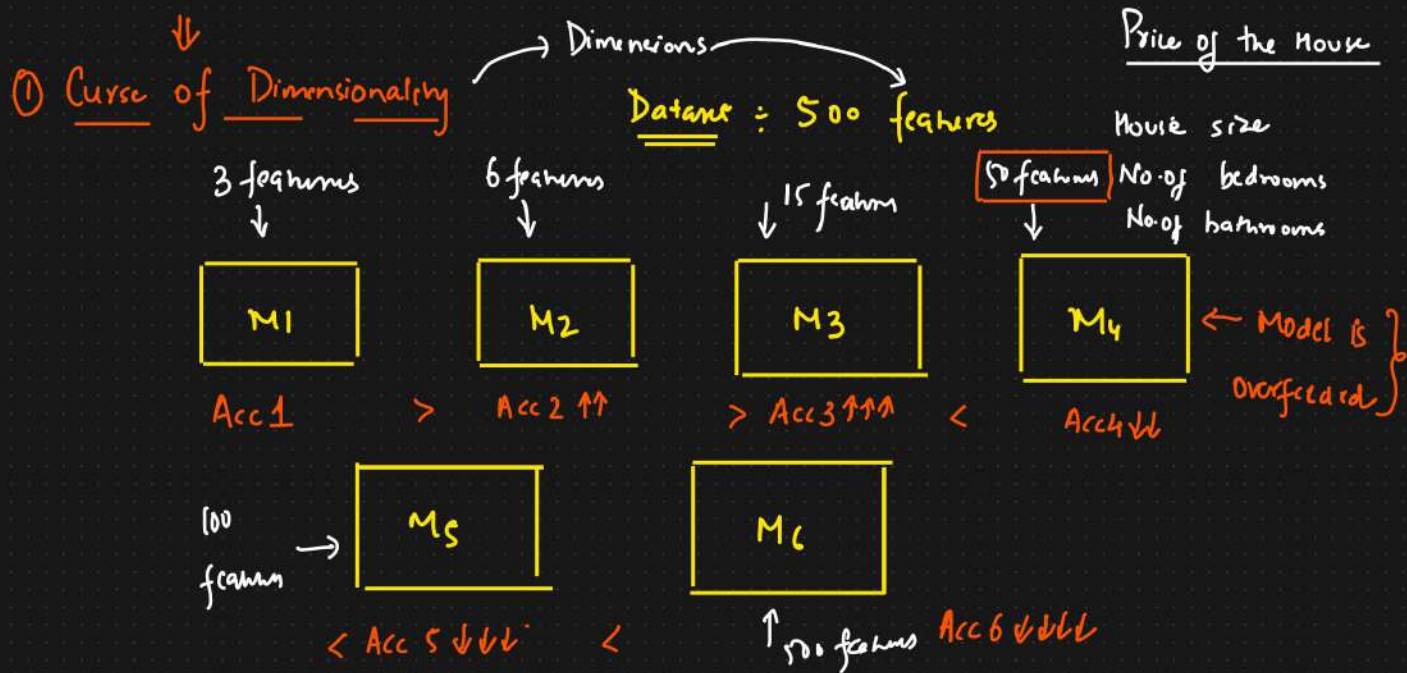
↳ New Product ← Discount



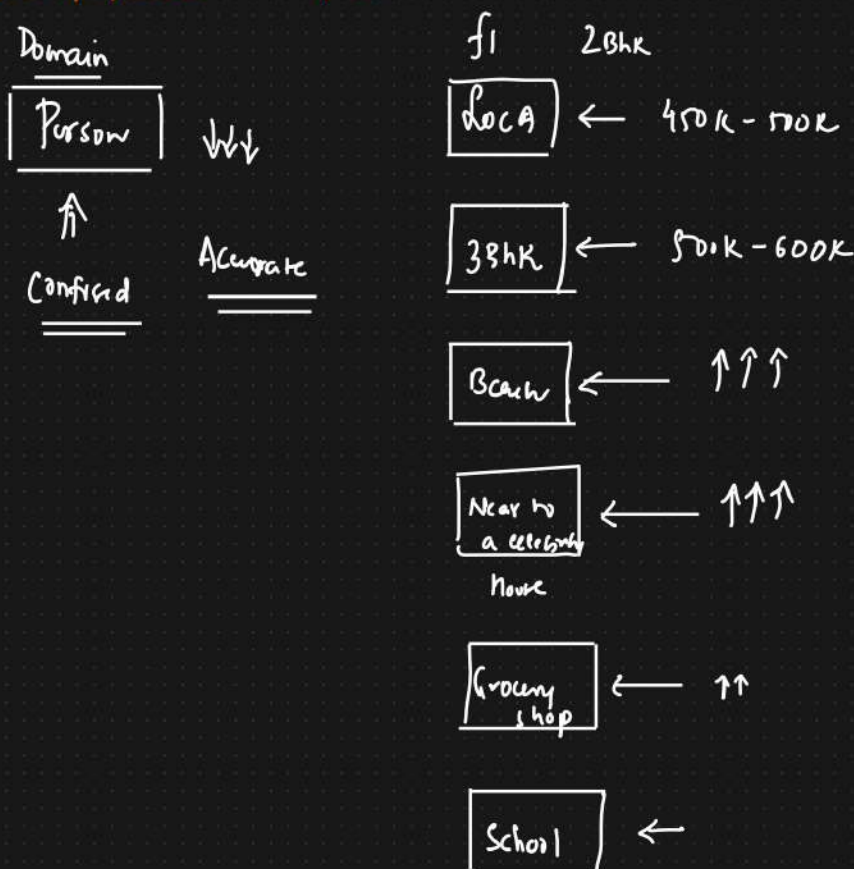
Unsupervised ML

- ① K Means Algorithm
 - ② Hierarchical Clustering
 - ③ DBScan Clustering
 - ④ Silhouette Scoring
- } validate

Principal Component Analysis (PCA) [Dimensionality Reductor]



② Model performance Degrade



original
f1 f2 f3 4p

Kernel ↓ Feature Extraction
D1 D2 O/p

Two different ways to remove curse of Dimensionality

① Feature Selection

② Dimensionality Reduction (PCA)

Imp features

Feature Extraction

Feature Selection Vs Feature Extraction

↳ Dimensionality Reduction

① Why Dimensionality Reduction?

- (*) Prevent → Curse of Dimensionality
- (*) Improve the performance of the model
- (*) Visualize the data → understand the data

3d 2d

100d

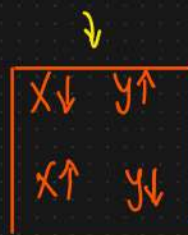
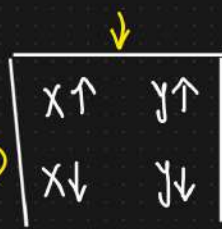
↓

3d or 2d

Feature Selection

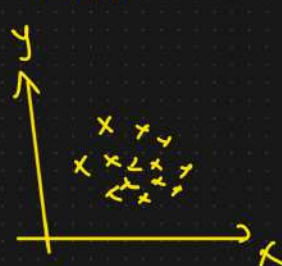
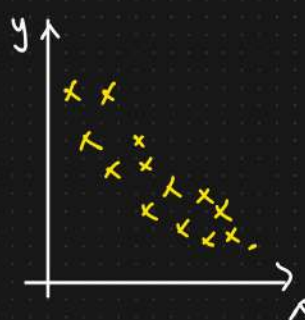
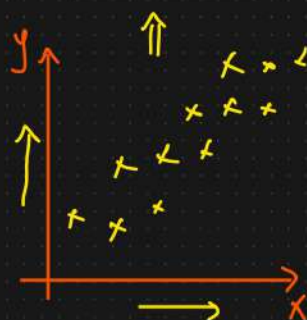
I/p o/p
X → y
 - -
 - -
 - -
 - -

+ve ⇒



← -ve

No relationship between X & y



$$\text{Cov}(X, Y) = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{N-1}$$

= +ve
= -ve
 ≈ 0

≈ 0 {No Relationship}

-ve correlated

Pearson Correlation = $\frac{\text{Cov}(X, Y)}{\sigma_x \sigma_y} = [-1 \text{ to } 1]$

$\sigma_x \neq \sigma_y$

The more toward the value of +1 the

more +ve correlated X & Y is

Datant housing

Independent feature

→ DIP feature

✓ House
Size

Fountain
Size

Price

{Frequency Selection}

Draft

{Linear Relationship}

Covergence = +ve
-ve

Price of
house $\uparrow$

$$0 - 0.2f$$

20 ↓

Fountain
Size

Feature Extraction

Domain Expertise

+ve
-ve

Price

Price

2 features $\rightarrow$ 1 feature

Dimensionality Reduction

Room Size

No. of Rooms 1

Price

Price

↓ Transformation To extract New feature

→ House Size

Price

PCA Geometric Intuition

{ Dimensionality Reduction }

Housing Dataset

Size of House | No. of Room | Price $\uparrow$ DIP

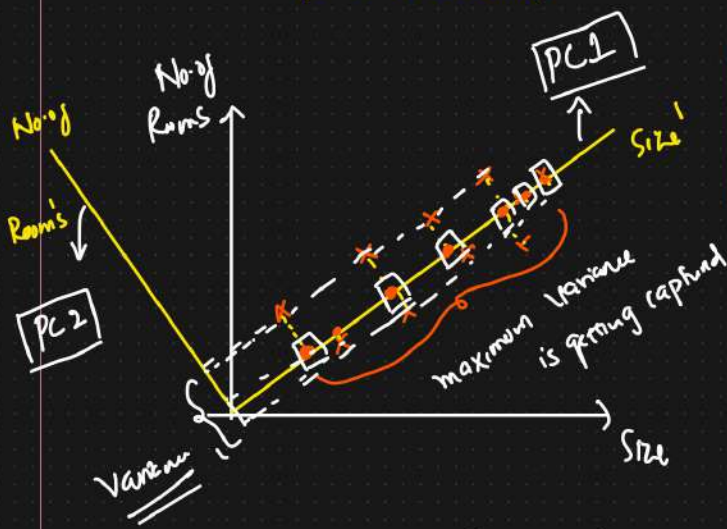
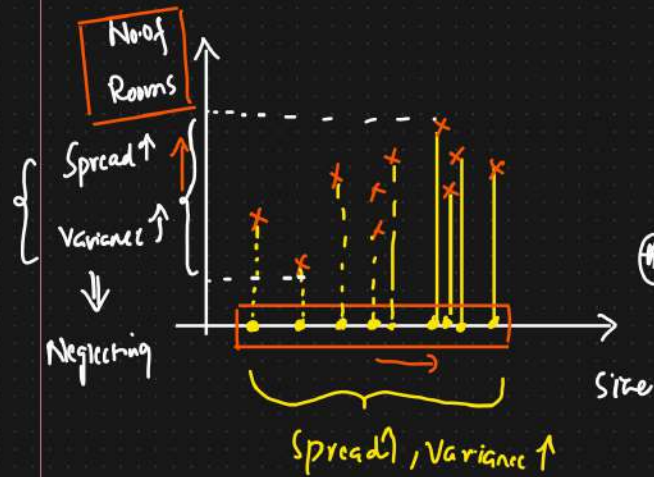
PCA

2 dimension $\rightarrow$ 1 dimension

2D $\rightarrow$ 1D

(*) loss of information (No. of Rooms)

Feature Extraction



Eigen decomposition on Matrix

Transformation

2 Dimension

PC1, PC2

$\text{Var}(\text{PC1}) > \text{Var}(\text{PC2})$

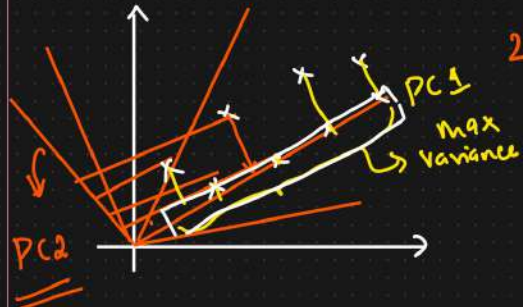
3 Dimension

PC1, PC2, PC3

$\text{Var}(\text{PC1}) > \text{Var}(\text{PC2}) > \text{Var}(\text{PC3})$

2D $\rightarrow$ 1 Dimension

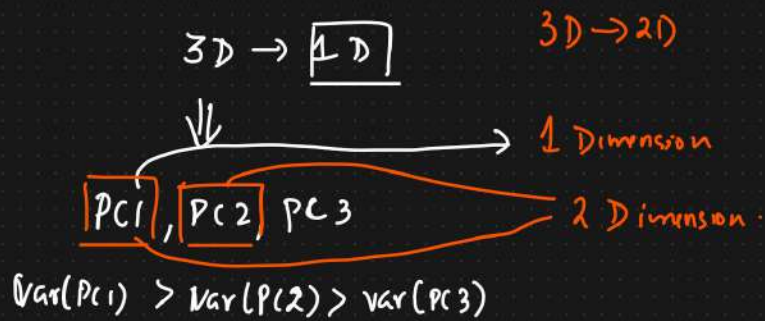
Much Information is not lost



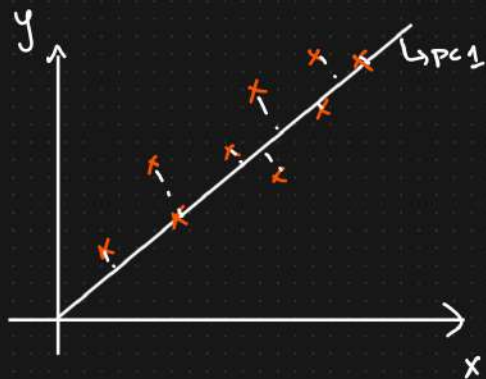
2D $\rightarrow$ 1D

2 Best Principal Component

To get the best Principal Component which captures maximum variance

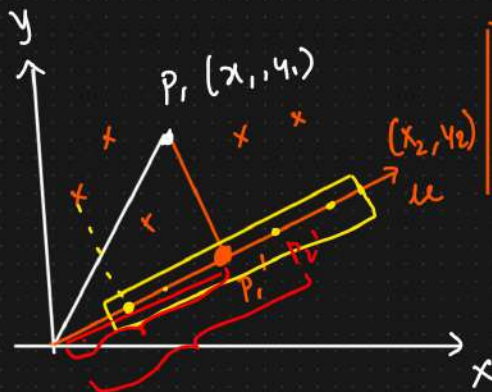


Maths Intuition behind PCA Algorithm



2D $\rightarrow$ 1D

- ① Projection $\checkmark$
- ② Cost function $\rightarrow$ Variance $\checkmark$

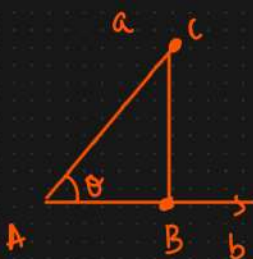


$$\text{Proj}_{P_1} u = \frac{P_1 \cdot u}{\|u\|}$$

$\|u\| = 1 \rightarrow$ unit vector

$$\text{Proj}_{P_1} u = P_1 \cdot u \Rightarrow \text{Scalar value}$$

$\downarrow$
 P_1'



$$P_0', P_1', P_2', P_3', P_4', \dots, P_n'$$

$\downarrow$
Scalar values
 $\downarrow$
Variance

$$p_0', p_1', p_2', p_3', p_4', \dots, p_n'$$

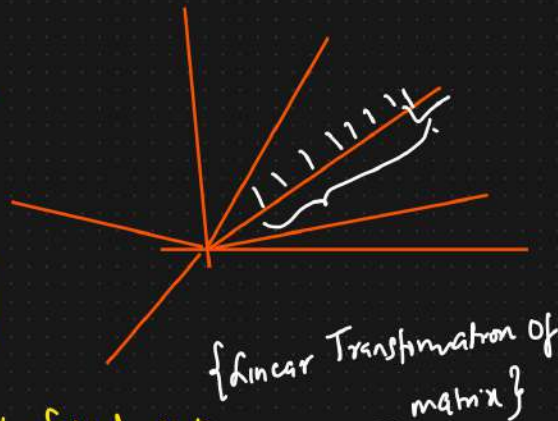
↓

$$x_0', x_1', x_2', x_3', x_4' \dots x_n'$$

$$\text{Max Variance} = \sum_{i=1}^n \frac{(x_i - \bar{x})^2}{n} \quad \left\{ \begin{array}{l} \text{Goal: Find the best} \\ \text{unit vector which} \\ \text{captures maximum variance} \end{array} \right\}$$

↓
Cost function

↓
Eigen vectors And Eigen values.
↓



① Covariance Matrix between features

② Eigen vectors and Eigen values will found out from this covariance matrix

$$A v = \lambda v$$

③ Eigen vector → Eigen value → magnitude of
the Eigen vector → Capture the maximum variance

Eigen vectors And Eigen values [Linear Transformation]



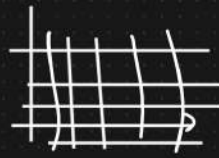
[Eigen decomposition of covariance Matrix]

↓

Eigen vector & Eigen values

$$[] * [v] = \lambda * v$$

↓
Eigen
value



$$A * v = \lambda * v$$

↑ v ↓

Eigen vector → Maximum magnitude



Eigen vector → Max Magnitude



Max Eigen vector



Best Principal Component → PC1

Principal Component



Max Var

Steps to calculate Eigen value and vectors

① Covariance of features

$$\begin{bmatrix} X & Y \end{bmatrix} \quad Z$$

↓
X'

$$\text{Cov}(X, Y) = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{N-1}$$

2x2 X Y

$$A = \begin{matrix} & \begin{matrix} X & Y \end{matrix} \\ \begin{matrix} X \\ Y \end{matrix} & \begin{bmatrix} \text{Var}(X) & \text{Cov}(X, Y) \\ \text{Cov}(Y, X) & \text{Var}(Y) \end{bmatrix} \end{matrix}$$

$$\text{Cov}(X, X) = \text{Var}(X)$$

$$\text{Cov}(Y, Y) = \text{Var}(Y)$$

	X	Y	Z
X			

y			
x			

f_1 & f_2

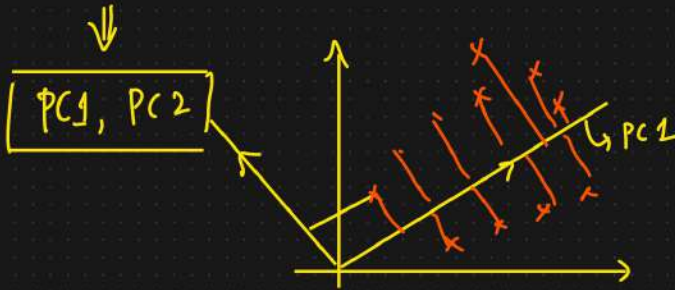
$\lambda_1, \lambda_2, \lambda_3$

$$A \cdot v = \lambda \cdot v$$

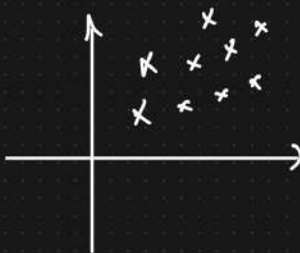
λ_1 λ_2

↓ ↓
PC1 PC2

$\lambda_1, \lambda_2 \rightarrow$ Eigen values



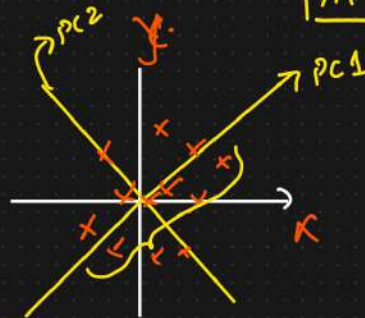
④



2D $\rightarrow$ 1D

$|\lambda_1| \Rightarrow$ magnitude of the Eigen vector

① Standardize the data



② Covariance Matrix of x & y

$$A = \begin{matrix} & \begin{matrix} x & y \end{matrix} \\ \begin{matrix} x \\ y \end{matrix} & \begin{bmatrix} \text{Var}(x) & \text{Cov}(x,y) \\ \text{Cov}(y,x) & \text{Var}(y) \end{bmatrix} \end{matrix} \quad 2 \times 2$$

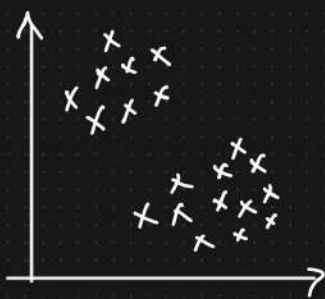
③ Find out Eigen vectors And value

$$A v = \lambda v$$

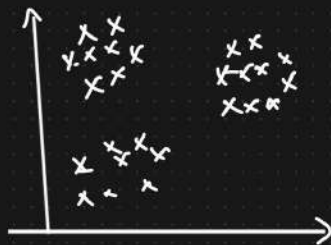
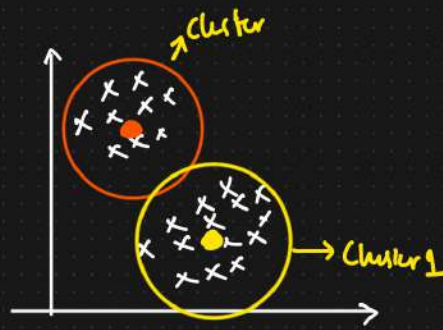
$\lambda_1, \lambda_2 \Rightarrow$ Eigen values.

↓ ↓
PC1 PC2

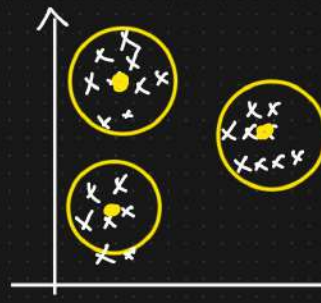
K Means Clustering Algorithm



K Means
 $\Rightarrow$



$\Rightarrow$



Eucledian Distance }
OR
MANHATTAN Distance }

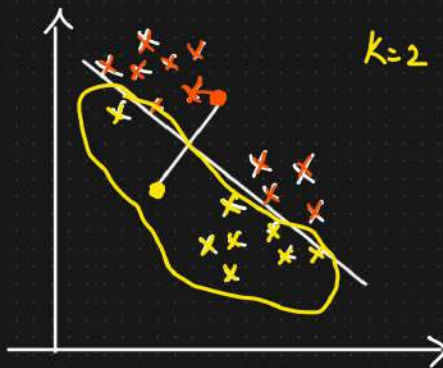
$K=2$

K Mean
 $\Rightarrow$

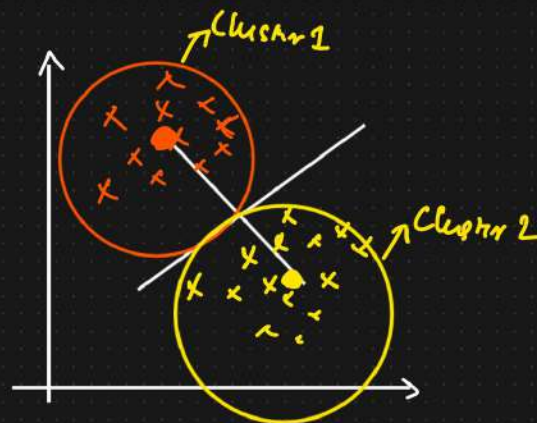
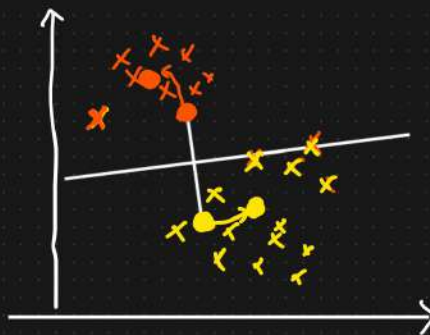
① Initialize Some $K \rightarrow$ centroids

② Points that are nearest to the centroids $\rightarrow$ Group

③ Move the centroids $\rightarrow$ Average



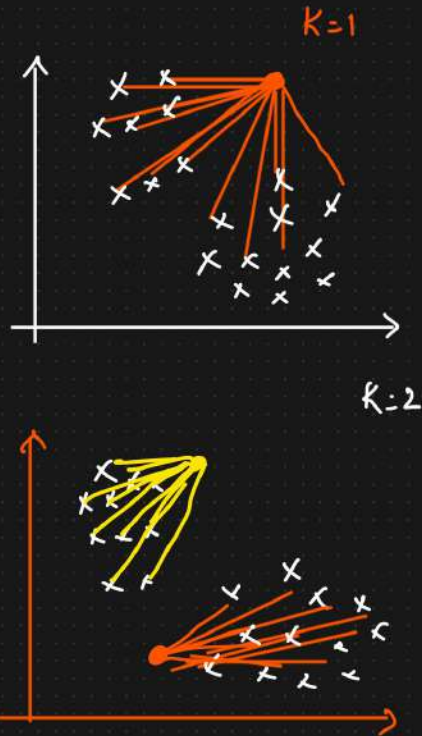
$K=2$



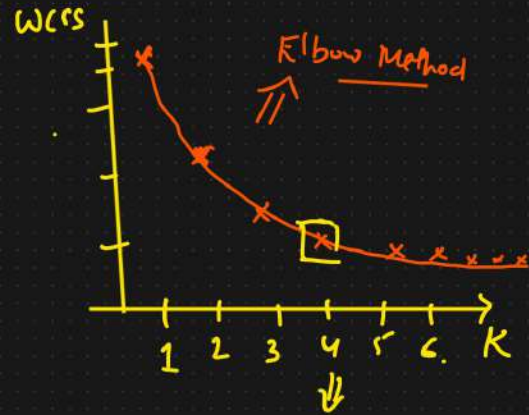
How do we select the K value?

WCSS = within cluster sum of squares

Initialize $K=1$ to 20

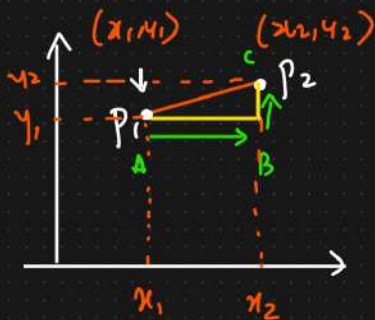


$$WCSS = \sum_{i=1}^n \left(\text{distance between points to nearest centroid} \right)^2$$



WCSS $\downarrow$

⑧ Euclidean Distance



$$\text{Euclidean dist} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

$$\text{Manhattan dist} = |x_2 - x_1| + |y_2 - y_1|$$

IRON MAN $\rightarrow$ U.S

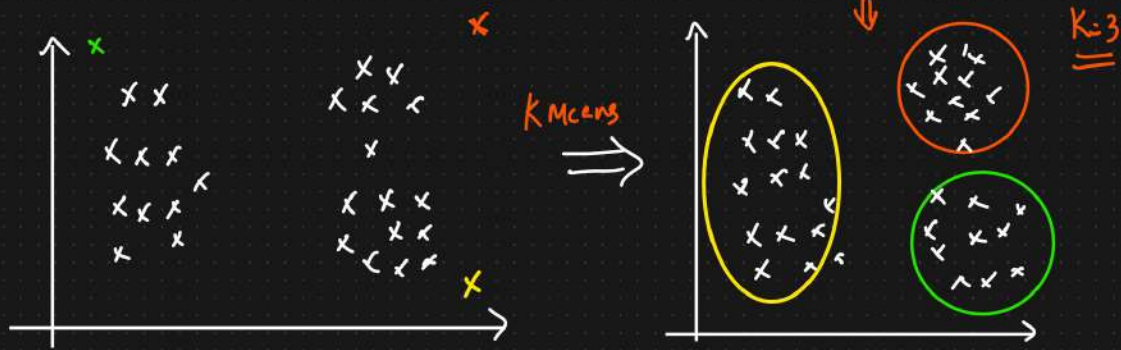
Air Traffic



$\Rightarrow$ Manhattan distance

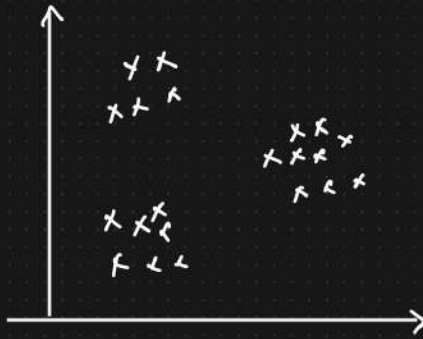


Random Initialization TRAP (Kmeans++)

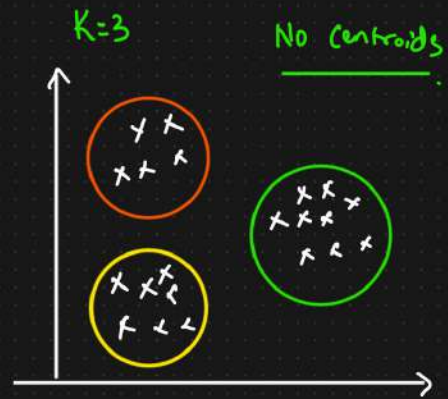


Kmeans++ Initialization Technique

Hierarchical Clustering

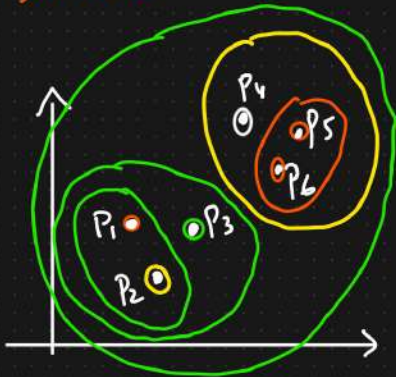


⇒ Hierarchical Clustering



HC

- ① Agglomerative
 - ② Divisive
- ⇒ Geometric Intuition.



Steps

- ① For each point initially will consider it as a separate cluster
- ② Find the nearest point and create a new cluster
- ③ Keep on doing the same process until we get a single cluster

Dendrogram

No. of clusters

Euclidean Distance

Distance

Threshold

K=2

↑↑↑↑

K=4

Dendrogram

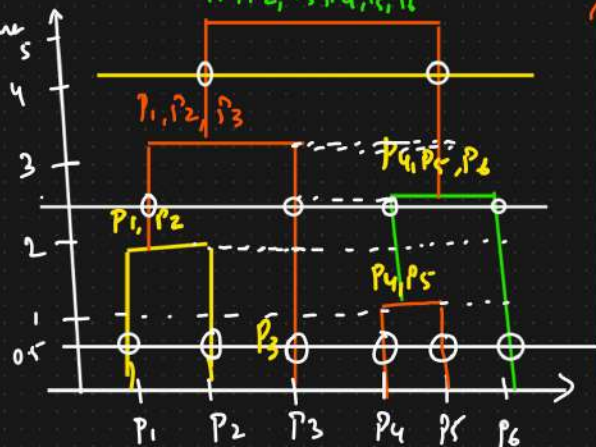
K=2

K=2

Agglomerative

Divisive

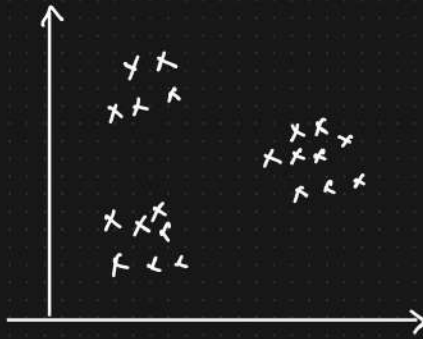
Euclidean Distance



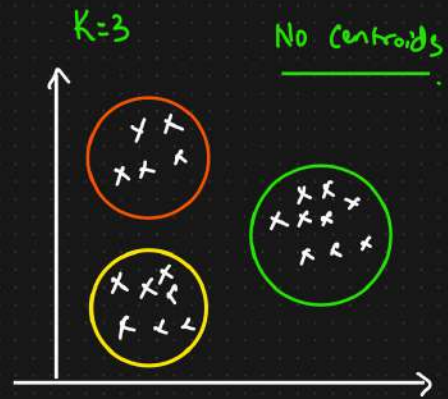
(*) Select the longest vertical line such that }
no horizontal line passes through it }

Threshold {Euclidean Distance} .

Hierarchical Clustering

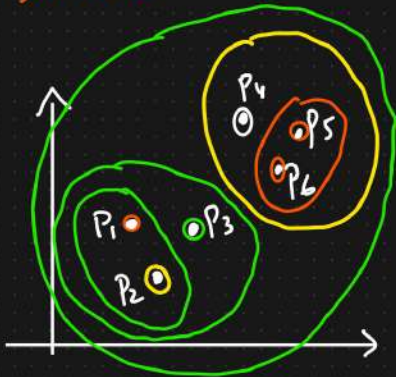


⇒ Hierarchical Clustering



HC

- ① Agglomerative
 - ② Divisive
- ⇒ Geometric Intuition.



Steps

- ① For each point initially will consider it as a separate cluster
- ② Find the nearest point and create a new cluster
- ③ Keep on doing the same process until we get a single cluster

Cosine Similarity

No. of clusters

Dendrogram

Euclidean Distance

Distance

Threshold

K=2

↑↑↑↑

K=4

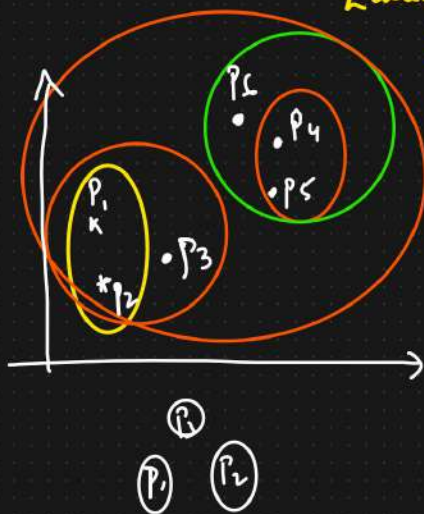
Dendrogram

K=2

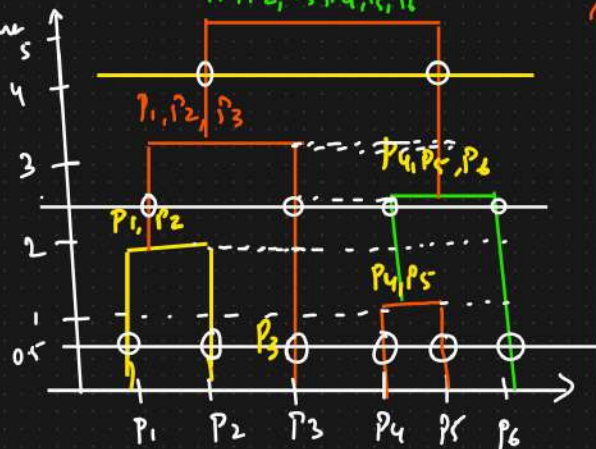
K=2

Agglomerative

Divisive



Euclidean Distance



- (*) Select the longest vertical line such that }
no horizontal line passes through it }

Threshold {Euclidean Distance}.

K Means Vs Hierarchical Clustering

Scalability And Flexibility

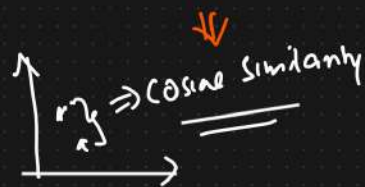
① Dataset size $\rightarrow$ Huge $\rightarrow$ K Means

Small $\rightarrow$ Hierarchical Clustering



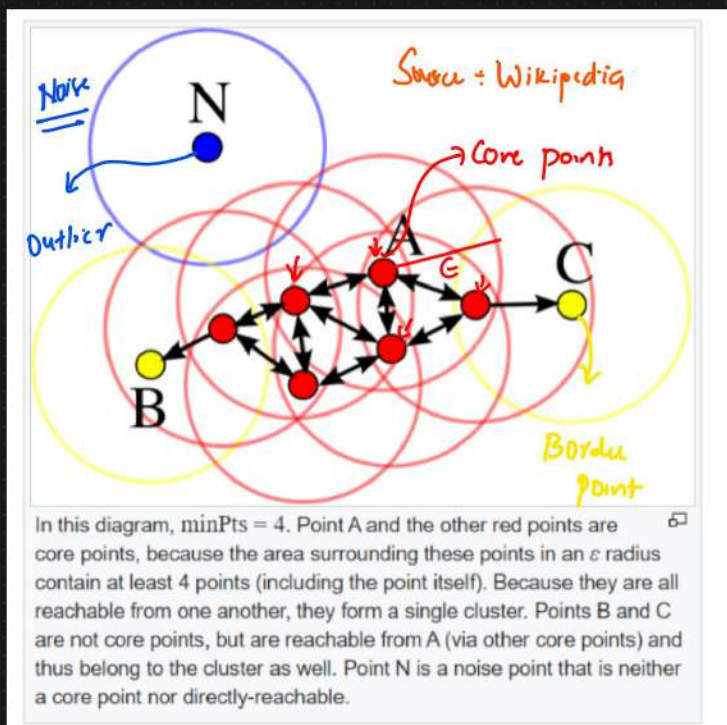
② Kmean $\rightarrow$ Numerical data

Hierarchical clustering $\rightarrow$ Variety of data.



③ Centroids $\rightarrow$ Elbow method $\rightarrow$ No. of centroids
 $\rightarrow$ No. of clusters

DBSCAN CLUSTERING.



- → Core point
 - → border point
 - → Outlier
- } Non linear Clustering

$\text{minpts} = 4$ $\epsilon = \text{radius}$

Core point

$\text{minpts} = 4$

① No. of points within the ϵ should be greater ≥ 4

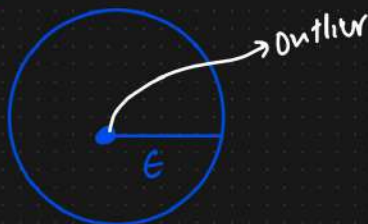


Border point

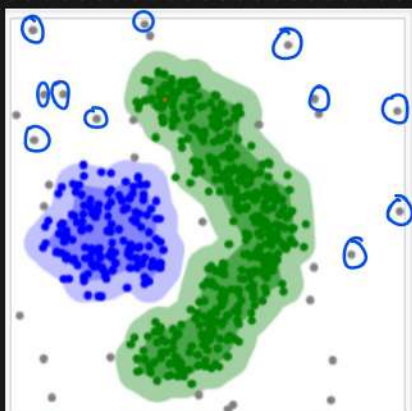
no. of data points within this radius will be less than minpts



Outlier (Noise)



Some Examples after we apply DBSCAN clustering



DBSCAN can find non-linearly separable clusters. This dataset cannot be adequately clustered with k-means or Gaussian Mixture EM clustering.



The left image depicts a more traditional clustering method that does not account for multi-dimensionality. Whereas the right image shows how DBSCAN can contour the data into different shapes and dimensions in order to find similar clusters.

Silhouette (Clustering)

①

$a(i)$



For data point $i \in C_I$ (data point i in the cluster C_I), let

$$a(i) = \frac{1}{|C_I| - 1} \sum_{j \in C_I, i \neq j} d(i, j) \quad \checkmark$$

be the mean distance between i and all other data points in the same cluster, where $|C_I|$ is the number of points belonging to cluster i , and $d(i, j)$ is the distance between data points i and j in the cluster C_I (we divide by $|C_I| - 1$ because we do not include the distance $d(i, i)$ in the sum). We can interpret $a(i)$ as a measure of how well i is assigned to its cluster (the smaller the value, the better the assignment).



$a(i)$

②

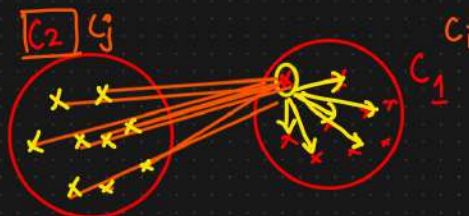
We then define the mean dissimilarity of point i to some cluster C_J as the mean of the distance from i to all points in C_J (where $C_J \neq C_I$).

For each data point $i \in C_I$, we now define

$$b(i) = \min_{J \neq I} \frac{1}{|C_J|} \sum_{j \in C_J} d(i, j) \Rightarrow b(i)$$

$$a(i) < b(i)$$

to be the *smallest* (hence the **min** operator in the formula) mean distance of i to all points in any other cluster, of which i is not a member. The cluster with this smallest mean dissimilarity is said to be the "neighboring cluster" of i because it is the next best fit cluster for point i .



$a(i)$

②

Silhouette Score

We now define a *silhouette* (value) of one data point i

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \text{ if } |C_I| > 1$$

$$[-1 \text{ to } 1]$$

and

$$s(i) = 0, \text{ if } |C_I| = 1$$

Which can be also written as:

$$s(i) = \begin{cases} 1 - a(i)/b(i), & \text{if } a(i) < b(i) \\ 0, & \text{if } a(i) = b(i) \\ \underline{b(i)/a(i) - 1}, & \text{if } a(i) > b(i) \end{cases}$$

From the above definition it is clear that

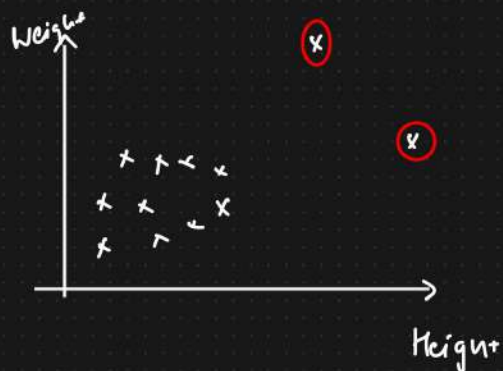
$$\{-1 \leq s(i) \leq 1\}$$

More near to 1 better

Clustering model we
have created

Anomaly Detection [To detect Outliers]

→ play a important Role

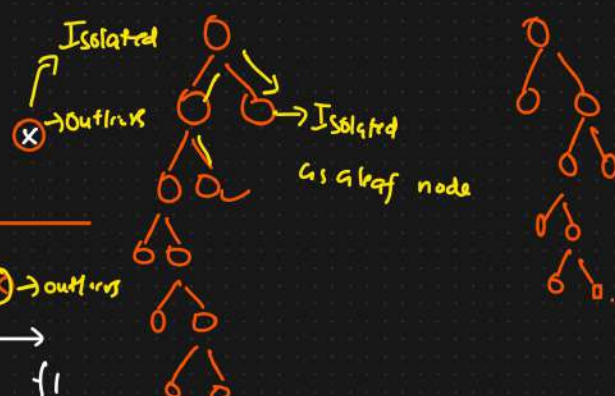
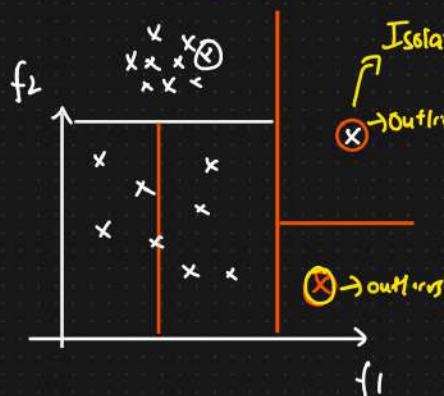


IPL	
1	15
2	10
3	12
4	100

> 36

1) Isolation Forest [Decision Trees] Many Trees Isolated Trees

f1	f2	f3	f4
-	-	-	-
-	-	-	-
-	-	-	-
-	-	-	-



Anomaly score

Mathematical Formula : Compute anomaly score for a new point

$$S(x, m) = 2^{-\frac{E(h(x))}{c(m)}}$$

m = no. of data points
x = data point

$E(h(x))$ = Average search depth for x from the isolate tree.

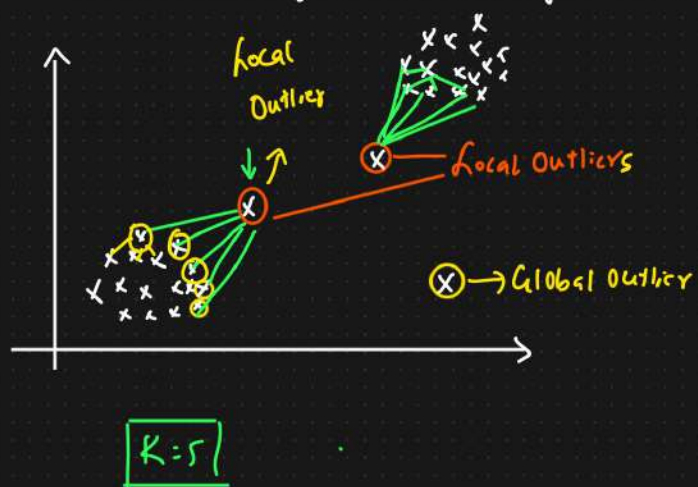
$c(m)$ = Average depth of h(n)

[Threshold > 0.5]

$E(h(x)) < c(m) \Rightarrow S(x, m) \approx 1 \Rightarrow$ Anomaly score $\Rightarrow$ Outliers

$E(h(x)) > c(m) \Rightarrow S(x, m) \approx 0.5 \Rightarrow$ Normal data point.

Local Outlier factor Anomaly Detection.



- ① Local Outlier
 - ② Global Outlier
- $[LOF] \Rightarrow LOF \text{ Score}$

K Nearest Neighbour

↓

Local Density